KẾ HOẠCH TỔNG THỂ M3 — KẾT HỢP DỮ LIỆU THỊ TRƯỜNG VÀ TÀI CHÍNH, BACKTEST, DASHBOARD VÀ BÁO CÁO TÁI LẬP

Dự án: DELTA — Phân cụm động lượng điều chỉnh theo rủi ro cho cổ phiếu Việt Nam
Kho mã nguồn: `trongnguyen1010/Risk-Adjusted-Momentum-Clustering`
Nhánh M3 tham chiếu: `feature/indicators-v1`
Nhánh M2/handoff tham chiếu: `m2-clustering-experiment`
Phiên bản kế hoạch: cập nhật output contract + input contract theo GitHub và các quyết định của chủ dự án ngày 26/10/2026; bản rà soát 03/11/2026 (xem "Ghi chú rà soát 03/11/2026" ngay bên dưới)
Trạng thái: KẾ HOẠCH CHI TIẾT — ENGINE/TARGETS-ONLY ĐÃ CÓ; CHƯA CHẠY OFFICIAL M3 BACKTEST CÓ RETURN
Phạm vi: M3-A Danh mục và backtest + M3-B Dashboard + M3-C Báo cáo và khả năng tái lập

GHI CHÚ RÀ SOÁT 03/11/2026

Bản này sửa trên commit `f507a3e`. Các thay đổi gồm làm rõ hoặc bổ sung; không đổi methodology đã khóa ngoài các điểm đánh dấu **CẦN OWNER XÁC NHẬN**.

#	Mục	Thay đổi	Lý do
R1	3.1.5, 5.2.A, 36.1	`cutoff(t) = max(30/04/(t+1), ngày usable theo Q-date)` — **CẦN OWNER XÁC NHẬN**	Với cutoff 30/04 cố định, FY < 2022 (usable sau 30/06/(t+1)) không có báo cáo nào để fit → B-PIT không xây được cho Development
R2	3.4.1, 5.2.A, 36.3	Thứ tự Financial cluster (median ROE, tie D/E) tính trên tập fit B-PIT tại cutoff và đóng băng trong artifact	Tránh tính profile bằng báo cáo chưa usable tại decision_date
R3	3.9, 4, 36.4	Segment boundary chuyển ĐỀ XUẤT → ĐÃ KHÓA; ghi hệ quả của `segment_end_date` Development = 2023-12-29	Đầu tài liệu và Mục 36 đã ghi là khóa; Mục 3.9/Mục 4 chưa sync
R4	3.5.3, 3.7	`N_candidates` đếm SAU khi loại `FINANCIAL_CRITERIA_MISSING`; N_min và `n_select` dùng cùng một N	Trước đây không nói rõ, có thể mở vị thế khi thực tế < 10 mã được chấm điểm
R5	3.8.6, 10.5.1	Thiếu dữ liệu trần/sàn → fail-closed, flag `LIMIT_DATA_MISSING`	Tránh mặc định khớp lệnh khi không kiểm tra được Q12
R6	3.4.7, 3.5.3, 5.3, 16.10	Persist `universe_audit_<strategy>.csv`	Cần cho báo cáo tỷ lệ loại theo lý do, giải thích `UNIVERSE_TOO_SMALL`, và cho Notebook 01/02 của M3-C

R7	4, 31, 36	M2 K=2: chỉ báo cáo số ứng viên, không chạy backtest — **CẦN OWNER XÁC NHẬN**	Development 42/60 tháng < 10 ứng viên → backtest gần như luôn `UNIVERSE_TOO_SMALL`
R8	31 (Giai đoạn 0, 3), 36.4	ADR là bước đầu tiên; chạy lại feasibility (chỉ đếm) với B-PIT trước LOCKED	Các số Va (13/60 tháng < 10) đều đo trên B v4

CẬP NHẬT QUYẾT ĐỊNH M3-A

Các decision M3-A đã được owner phê duyệt có ưu tiên cao hơn các proposal hoặc mô tả cũ trong tài liệu.

Từ ngày 02/11/2026, decision queue Q1–Q17 và Q-date được coi là đã khóa về methodology theo nội dung chi tiết tại Mục 3, Mục 13, Mục 17 và Mục 36. Việc methodology đã khóa không đồng nghĩa implementation/input đã READY; các dependency kỹ thuật vẫn phải PASS trước official run.

Các điểm chính hiện hành:

- A v4, k=3, là Market model chính thức; M2 K=2 giữ làm legacy/research control, chỉ báo cáo số ứng viên, không chạy backtest (R7).
- Financial clustering chính thức cho official M3 là B-PIT; hiện CHƯA READY cho tới khi build + audit + pin hoàn tất. Cutoff B-PIT = max(30/04/(t+1), ngày usable theo Q-date) (R1, cần owner xác nhận).
- Market và Financial là hai hệ clustering độc lập; M3 chỉ kết hợp ở tầng strategy/portfolio.
- Combined Candidate Universe sử dụng Market leading cluster ∩ Financial not-weakest clusters sau các PIT/eligibility gate.
- ILLIQUID bị loại khỏi traded universe của M3.
- MC1 = động lượng 6 tháng (mom_126); MC2 = biến động giảm 63 phiên (downside_vol_63, thấp hơn tốt hơn); MC3 = thanh khoản/giá trị giao dịch 21 phiên (liquidity_21).
- FC1 = ROE; FC2 = Piotroski F-score; FC3 = Z-score; missing một phần → percentile 0.5 + flag, thiếu cả ba → loại khỏi Financial scoring.
- Combined score = 0.5 × S_market + 0.5 × S_fin; hòa điểm → liquidity_21 cao hơn → ticker A→Z.
- N_min = 10 cho Combined, Market-only và Financial-only; không áp cho Equal-weight/Momentum-only theo strategy contract hiện tại.
- Financial runtime PIT: lọc usable_from_date <= decision_date, sau đó chọn max(fiscal_year).
- FY >= 2022 dùng disclosure date CafeF đã audit; FY < 2022 dùng lag rule phiên đầu tiên sau 30/06 năm t+1; mốc 30/04 chỉ là sensitivity check.
- Financial staleness: FY t hết hiệu lực khi FY t+1 usable hoặc vượt hard cap 18 tháng sau period_end, điều kiện nào đến trước.
- Execution sử dụng phiên giao dịch hợp lệ kế tiếp; volume 0 chặn hai chiều; close = ceiling chặn BUY; close = floor chặn SELL.
- Q17 dùng derived adjusted-price repair cho one-session glitch, không sửa source data.
- Main return basis = adj_close; VNINDEX là price index và phải disclosure basis mismatch.
- Secondary price-only = NOT_READY do chưa có corporate-action readiness; không block main

adj_close backtest.

- Main transaction cost = 0, slippage = 0. Cost sensitivity 15/30 bps chuyển sang PHẦN MỞ RỘNG.
- Segment boundary dùng strict isolation: Target vẫn tạo/lưu nếu decision_date nằm trong segment; nếu execution_date > segment_end thì không execution và không dùng giá ngoài segment.
- Missing valuation price dùng PRICE_STALE. DELISTED_EXIT là PHẦN MỞ RỘNG, chưa implement.
- Development = 60 snapshots 2019-01-31 → 2023-12-29; Holdout = 24 snapshots 2024-01-31 → 2025-12-31.
- Holdout chỉ được coi là sạch đối với portfolio-rule/return-based evaluation; không tuyên bố hoàn toàn unseen đối với toàn bộ upstream model-selection history.
- VNINDEX là benchmark chính thức.
- Final Evaluation bắt đầu từ snapshot cuối tháng 01/2026; exact end date phải được khóa sau readiness audit và trước khi xem Final performance. Sharpe chỉ báo cáo ở Final.

Các decision trên phải được sync vào docs/DECISIONS.md, config và code tương ứng trước official run. Chi tiết trạng thái xem Mục 36 — Trạng thái hiện tại.

---

0. CÁCH ĐỌC TÀI LIỆU

Tài liệu dùng các trạng thái sau:

- **ĐÃ KHÓA / KẾ THỪA:** đã có căn cứ từ M1, M2, kho mã nguồn hoặc quyết định trước đó; không được tự đổi.
- **ĐÃ ĐƯỢC CHỦ DỰ ÁN DUYỆT:** quyết định đã được chốt cho M3.
- **ĐỀ XUẤT:** phương án triển khai hợp lý nhưng chưa được coi là quyết định cuối.
- **CẦN QUYẾT ĐỊNH CỦA CHỦ DỰ ÁN / MENTOR:** chưa đủ căn cứ để tự chọn; nhiệm vụ liên quan phải dừng nếu quyết định này ảnh hưởng trực tiếp đến kết quả.

Mỗi nhiệm vụ đều có:

1. Mục tiêu;
2. Trạng thái / căn cứ quyết định;
3. Đầu vào bắt buộc;
4. Đầu ra mong đợi;
5. Công cụ / tệp liên quan;
6. Các bước thực hiện theo thứ tự;
7. Kiểm tra / xác nhận;
8. Điều kiện chuyển bước;
9. Bàn giao sang nhiệm vụ tiếp theo.

Không được bỏ qua điều kiện chuyển bước chỉ vì mã nguồn "vẫn chạy được".

---

1. TÀI LIỆU NGUỒN BẮT BUỘC

M3 ưu tiên các tài liệu và decision được cập nhật gần nhất trên nhánh M3 hiện hành.

Khi các tài liệu có nội dung khác nhau, áp dụng thứ tự ưu tiên:

```
Owner decision / DECISIONS.md
        ↓
M3_PLAN.md
        ↓
M3_MENTOR_QUESTIONS.md
        ↓
config đã LOCKED
        ↓
tài liệu kế hoạch tổng hợp này
```

Tài liệu kế hoạch tổng hợp này không được tự ghi đè owner decision hoặc M3 decision mới hơn trên Git.

1.1. `docs/DECISIONS.md`

Đây là nơi ghi các quyết định đã được owner phê duyệt.

Các decision M3 mới hơn phải được ưu tiên hơn proposal cũ trong plan.

Hiện đã có:

```
D-DIR-01 — 26/10/2026
```

Theo D-DIR-01 hiện đang có trên Git:

- A v4 là Market clustering model chính thức cho M2/M3;
- B v4 được ghi là Financial clustering model hiện hành tại thời điểm 26/10/2026;
- M2 K=2 được giữ làm legacy/research control;
- historical artifacts cũ không bị xóa;
- các giới hạn của model phải được báo cáo minh bạch.

Owner decision ngày 02/11/2026 đã chọn B-PIT cho official M3 để xử lý timing leakage. Decision mới này phải được ghi thành ADR/decision mới trên Git, và đây là bước đầu tiên của Giai đoạn 0 (Mục 31) vì theo thứ tự precedence ở trên, `DECISIONS.md` có ưu tiên cao nhất; cho tới khi Git sync, D-DIR-01 vẫn là historical Git record nhưng không được dùng để phủ quyết owner decision mới hơn.

Một nội dung đã được ghi thành owner decision trong `DECISIONS.md` không được tiếp tục trình bày như một câu hỏi chưa chốt trong kế hoạch M3.

1.2. `docs/M3_PLAN.md`

Đây là tài liệu M3 chính để xác định:

- phạm vi M3-A, M3-B, M3-C;

- Development / Holdout / Final Evaluation;
- strategy definitions;
- Candidate Universe;
- portfolio construction;
- execution assumptions;
- benchmark;
- decision queue Q1–Q17;
- trình tự triển khai M3.

Các nội dung trong `M3_PLAN.md` có trạng thái proposal/open không được tự coi là đã khóa.

1.3. `docs/M3_MENTOR_QUESTIONS.md`

Tài liệu này là nguồn của các proposal Q1–Q17/Q-date trước khi owner chốt.

Từ decision ngày 02/11/2026, các proposal được chọn trong decision queue đã trở thành methodology đã khóa theo kế hoạch này. Các câu trả lời/decision vẫn phải được ghi đồng bộ vào `docs/DECISIONS.md` /ADR và config trên Git để repository trở thành authority thực thi.

Phải phân biệt:

```
owner decision đã chốt
≠
config/code đã sync
≠
runtime/input đã READY
```

Official backtest chỉ được chạy khi cả decision, implementation/config và readiness gate tương ứng đều PASS.

1.4. `configs/experiments/m3_strategy_v1.json`

Đây là config thực thi của strategy M3.

Decision queue đã được owner khóa về methodology, nhưng config Git hiện tại vẫn cần được sync theo các quyết định mới. Nếu config checkout hiện vẫn có:

```
status = PROPOSED
```

thì đây là implementation/config-sync blocker, không còn là governance blocker.

Trước official Development/Holdout run, config phải tối thiểu phản ánh:

```
market_model = A v4, k=3
financial_model = B-PIT đã pin
market_rule = leading
financial_rule = not_weakest
n_min = 10
```

```
stale_months = 18
combined_weight_market = 0.5
top_share = 0.3
main price basis = adj_close
main transaction cost = 0
main slippage = 0
segment boundary = strict isolation
```

Các scenario 15/30 bps không thuộc main config/run hiện tại; chuyển sang phần mở rộng.

Chỉ khi config đã sync, chuyển sang:

```
status = LOCKED
```

và config hash đã được pin thì mới được sử dụng cho official Development/Holdout run.

1.5. Tài liệu M3-B hiện hành

M3-B phải bám kiến trúc product/web hiện hành của repository:

```
web/frontend
→ Next.js

web/backend
→ FastAPI
```

M3-B là read-only product/presentation layer.

Không dùng tài liệu hoặc implementation dashboard legacy để ghi đè kiến trúc M3-B hiện hành.

1.6. Nguyên tắc precedence

Nếu phát hiện khác biệt giữa:

```
kế hoạch tổng hợp hiện tại
vs
Git M3 mới hơn
```

thì:

1. xác định trạng thái của nội dung trên Git;
2. nếu là owner-approved decision → cập nhật kế hoạch theo decision đó;
3. nếu chỉ là proposal → giữ trạng thái OPEN/PROPOSED;
4. không tự biến proposal thành decision;
5. không dùng tài liệu Git cũ hơn để ghi đè decision M3 mới hơn.

---

2. KIẾN TRÚC NGHIÊN CỨU M3

2.1. Hai hệ phân cụm độc lập

```
DỮ LIỆU THỊ TRƯỜNG
    ↓
Tiền xử lý dữ liệu thị trường
    ↓
Phân cụm thị trường
    ↓
K_market
    ↓
Nhãn phân cụm thị trường
```

và:

```
DỮ LIỆU TÀI CHÍNH
    ↓
Tiền xử lý dữ liệu tài chính
    ↓
Phân cụm tài chính
    ↓
K_financial
    ↓
Nhãn phân cụm tài chính
```

M3 không được làm:

```
Đặc trưng thị trường thô
+
Đặc trưng tài chính thô
→ gộp thành một bộ đặc trưng
→ phân cụm lại
→ chọn một K chung
```

2.2. Phần kết hợp nằm ở bước ra quyết định danh mục

```
Nhãn phân cụm thị trường tại ngày t
        +
Nhãn phân cụm tài chính mới nhất đã thực sự có tại ngày t
        ↓
Căn chỉnh theo thời gian và PIT
        ↓
Chọn Market leading cluster theo profile
        +
Chọn Financial allowed clusters = tất cả trừ weakest cluster
        ↓
Market Eligible Pool ∩ Financial Eligible Pool
```

```
        ↓
CANDIDATE UNIVERSE
        ↓
Tính 3 tiêu chí thị trường + 3 tiêu chí tài chính
        ↓
Áp dụng quy tắc xếp hạng kết hợp đã được khóa
        ↓
Chọn 30% số mã đứng đầu, làm tròn lên
        ↓
Phân bổ trọng số bằng nhau
        ↓
Mô phỏng giao dịch ở giá đóng cửa của phiên hợp lệ kế tiếp
        ↓
Backtest danh mục
```

Candidate Universe phải được tạo **trước** khi tính MC1–MC3 và FC1–FC3. Sáu tiêu chí dùng để xếp hạng các mã đã đủ điều kiện, không dùng để tự định nghĩa eligibility.

2.3. Vai trò của năm chiến lược

Chiến lược chính — Kết hợp thị trường và tài chính

Combined kết hợp Market và Financial theo PIT, eligibility, scoring và ranking đã khóa.

Candidate Universe:

```
Market Eligible Pool
∩
Financial Eligible Pool
```

Đối chứng 1 — Market-only: Chỉ dùng tín hiệu thị trường

Market-only dùng tín hiệu thị trường để lựa chọn/xếp hạng cổ phiếu.

Đối chứng 2 — Financial-only: Chỉ dùng tín hiệu tài chính

Financial-only dùng tín hiệu tài chính để lựa chọn/xếp hạng cổ phiếu.

Universe phải đồng thời thuộc tập mã có giao dịch tại thời điểm tương ứng.

Market leading cluster không được dùng làm điều kiện chọn mã.

Đối chứng 3 — Equal-weight

Equal-weight sử dụng toàn bộ universe có giao dịch theo contract của strategy.

Không sử dụng Market/Financial ranking và không chọn top 30%.

Toàn bộ mã hợp lệ được phân bổ equal weight.

Đối chứng 4 — Momentum-only

Momentum-only sử dụng universe có giao dịch và mom_126 hợp lệ.

Các mã được xếp hạng theo mom_126, chọn top 30% và phân bổ equal weight.

Financial signal/ranking không tham gia tạo Target Portfolio.

Đối chuẩn chính thức

M3 hiện chỉ sử dụng **VNINDEX** làm benchmark chính thức để so sánh với kết quả backtest của chiến lược Combined.

VNINDEX là benchmark, không phải strategy thứ sáu.

Năm strategy M3 hiện tại gồm:

```
combined
market_only
financial_only
equal_weight
momentum_only
```

BẢNG THUẬT NGỮ VÀ BIẾN CỐT LÕI M3-A

Bảng này giải thích các khái niệm xuất hiện xuyên suốt M3-A. Các công thức và biến kỹ thuật chi tiết của backtest được trình bày tại **Bảng các biến tính toán trong 4 bước M3-A** sau Mục 7.

Thuật ngữ / biến	Ý nghĩa	Dạng
`ticker`	Mã cổ phiếu.	Chuỗi
`decision_date`	Ngày strategy sử dụng thông tin đã hợp lệ tại thời điểm đó để lựa chọn cổ phiếu và tạo Target Portfolio.	Ngày
`execution_date`	Phiên giao dịch hợp lệ đầu tiên sau `decision_date`, dùng để thực hiện rebalance theo execution contract.	Ngày
`market_cluster`	Cụm thị trường của ticker tại `decision_date`, do Market clustering upstream tạo.	Nhãn cụm
`fiscal_year`	Năm tài chính mà Financial state tương ứng đại diện.	Năm
`usable_from_date`	Ngày sớm nhất Financial state của một `fiscal_year` được phép sử dụng trong M3 theo PIT.	Ngày
`selected_fiscal_year`	Năm tài chính được M3 chọn trong số các năm đã thỏa `usable_from_date <= decision_date`; theo implementation hiện tại là `max(usable_financial_years)`.	Năm
`financial_cluster`	Cụm tài chính của ticker tương ứng với `selected_fiscal_year`.	Nhãn cụm

`ILLIQUID`	Trạng thái upstream gắn cho mã có thanh khoản thấp. Rule Git hiện mô tả mã là kém thanh khoản khi trong cửa sổ 63 phiên có hơn 50% phiên không khớp lệnh hoặc giá trị giao dịch dưới 10 triệu đồng/ngày. Theo decision Q2 đã khóa, ILLIQUID bị loại khỏi traded universe của M3.	Trạng thái thanh khoản
`traded universe`	Tập ticker cơ sở được phép tham gia bước selection tại `decision_date`. Theo decision Q2 đã khóa, tập này được tạo từ Market snapshot và loại các ticker `ILLIQUID`. Đây là selection-time universe, không có nghĩa một lệnh BUY/SELL chắc chắn thực hiện được tại `execution_date`.	Tập ticker
`Market Eligible Pool`	Tập ticker thỏa các điều kiện của nhánh Market, gồm liquidity/eligibility và Market cluster rule tương ứng.	Tập ticker
`Financial Eligible Pool`	Tập ticker có Financial state hợp lệ theo PIT và thỏa các điều kiện Financial eligibility/cluster rule tương ứng.	Tập ticker
`Candidate Universe`	Tập ticker thực tế được một strategy đưa vào bước ranking/selection. Universe này phụ thuộc strategy; ví dụ Combined dùng `Market Eligible Pool ∩ Financial Eligible Pool`.	Tập ticker
`N_candidates` / `candidates`	Số ticker trong Candidate Universe của strategy tại kỳ tương ứng.	Số nguyên
`Target Portfolio` / `target`	Danh mục mà strategy muốn nắm giữ sau rebalance, biểu diễn bằng mapping `ticker → target_weight`.	Mapping
`Current Portfolio`	Danh mục thực tế đang được engine nắm giữ trước rebalance của kỳ hiện tại, gồm `cash`, `values[ticker]` và `marks[ticker]`.	State
`Actual Holdings`	Trạng thái danh mục thực tế sau khi các BUY/SELL executable đã được xử lý; có thể khác Target Portfolio do failed trade hoặc scaling.	State / output weight
`tradable(ticker, date, side)`	Kiểm tra tại `execution_date` xem một lệnh BUY hoặc SELL cụ thể có thực hiện được hay không theo price, volume và ceiling/floor rule. Khác với `traded universe`, vốn được dùng ở bước selection tại `decision_date`.	Boolean

Cần phân biệt:

```
traded universe
→ Bước 1
→ decision_date
→ ticker có được phép tham gia selection universe hay không

tradable(...)
→ Bước 2
```

```
→ execution_date
→ một lệnh BUY/SELL cụ thể có thực hiện được hay không
```

---

3. CÁC QUY TẮC

Phần này tập hợp và chuẩn hóa các quy tắc vận hành của M3 thành một hệ thống thống nhất, bao gồm: tính toàn vẹn thời gian và dữ liệu tài chính (PIT, Staleness), điều kiện thanh khoản, eligibility theo từng strategy, tiêu chí chấm điểm, quy tắc chọn top 30%, ngưỡng quy mô tối thiểu, quy tắc khớp lệnh thực thi và quy tắc kết thúc segment/run.

Các giá trị thuộc decision queue vẫn ở trạng thái OPEN/PROPOSED phải được phân biệt với rule đã được owner/mentor khóa. Việc một proposal đã được hiện thực trong code hoặc có mặt trong config không tự làm proposal đó trở thành official methodology.

3.1. Quy tắc thời gian và Point-in-Time (PIT)

3.1.1. Lịch ra quyết định hàng tháng

M3 dùng lịch ra quyết định hàng tháng dựa trên các mốc dữ liệu thị trường hợp lệ.

3.1.2. Dữ liệu tài chính có lịch công bố riêng

Dữ liệu tài chính có thể cập nhật theo quý, nửa năm hoặc năm và luôn có độ trễ công bố. Vì vậy không được ép mốc tài chính phải trùng ngày với mốc thị trường.

3.1.3. Ghép theo Financial state hợp lệ tại thời điểm quyết định

`decision_date` là ngày ra quyết định của M3.

Tại mỗi `decision_date = t` :

```
market_cluster
= nhãn phân cụm thị trường tại t

usable_financial_years
= mọi fiscal_year có usable_from_date <= decision_date

selected_fiscal_year
= max(usable_financial_years)

financial_cluster
= Financial state tương ứng với selected_fiscal_year
```

Nếu không có `fiscal_year` nào thỏa:

```
usable_from_date <= decision_date
```

thì:

```
FINANCIAL_NOT_AVAILABLE
```

Financial PIT của M3 thực hiện theo thứ tự:

```
usable_from_date <= decision_date
↓
lọc các fiscal_year đã usable
↓
selected_fiscal_year = max(fiscal_year)
↓
lấy Financial state tương ứng
```

Ý nghĩa của hai trường thời gian:

```
usable_from_date
→ xác định record tài chính của từng fiscal_year
  đã hợp lệ để sử dụng tại decision_date hay chưa

fiscal_year
→ xác định năm tài chính mới nhất
   trong số các năm đã được phép sử dụng
```

Ví dụ:

```
FY2022
usable_from_date = 2023-03-13

FY2023
usable_from_date = 2024-03-11
```

Tại:

```
decision_date = 2024-03-08
```

thì:

```
usable_financial_years = {2022}
selected_fiscal_year = 2022
```

Tại:

```
decision_date = 2024-03-11
```

thì:

```
usable_financial_years = {2022, 2023}
```

```
selected_fiscal_year = 2023
```

Không dùng `period_end` thay cho `usable_from_date` để xác định Financial data đã được phép sử dụng tại `decision_date` hay chưa.

3.1.4. Quy tắc xác định usable_from_date của BCTC năm — Q-date

Canonical rule:

- FY >= 2022: dùng disclosure date CafeF đã audit; usable_from_date là phiên giao dịch hợp lệ đầu tiên sau ngày công bố hợp lệ.
- FY < 2022: không dùng historical CafeF disclosure date làm canonical; BCTC năm t usable từ phiên giao dịch hợp lệ đầu tiên sau 30/06 năm t+1.
- Mốc 30/04 chỉ dùng cho sensitivity/robustness check riêng, không thay canonical rule 30/06.

Rule này ưu tiên tránh look-ahead. Với doanh nghiệp có fiscal year khác 31/12, lag rule có thể conservative hơn thực tế nhưng không được tự rút ngắn nếu chưa có authority dataset tốt hơn.

3.1.5. Financial clustering PIT — Q3

Official M3 sử dụng B-PIT thay cho việc dùng trực tiếp Financial B centers đã fit bằng toàn bộ báo cáo của fiscal year.

Với fiscal year t:

```
cutoff(t) = max(30/04/(t+1), usable_date_QDATE(t))
```

Trong đó `usable_date_QDATE(t)` là ngày usable chung theo Q-date (Mục 3.1.4):

```
FY >= 2022
→ cutoff(t) = 30/04/(t+1)
→ báo cáo nộp sau 30/04 (khoảng 9–15% mã-năm theo dữ liệu 2022–2025) được gán vào centers đã c

FY < 2022
→ mọi báo cáo usable từ phiên đầu tiên sau 30/06/(t+1)
→ cutoff(t) = phiên đầu tiên sau 30/06/(t+1)
→ B-PIT fit trên toàn bộ báo cáo FY t
```

Lý do: nếu giữ cutoff cố định 30/04, FY2017–FY2021 không có báo cáo nào có `usable_from_date <= cutoff`, nên B-PIT không fit được và Combined/Financial-only của Development bị chặn hoàn toàn. Với FY < 2022, fit tại phiên đầu sau 30/06 không tạo leakage vì trước ngày đó không decision nào được dùng FY t. Hệ quả cần ghi trong báo cáo: B-PIT chỉ khác B v4 ở FY >= 2022.

Trạng thái: làm rõ Q3 — CẦN OWNER XÁC NHẬN khi ghi ADR. Mốc 30/04 cho FY < 2022 có thể giữ làm sensitivity cùng kịch bản Q-date 30/04.

Tại cutoff:

```
chỉ dùng report có usable_from_date <= cutoff(t)
↓
```

```
fit preprocessing/scaling + clustering
↓
đóng băng model/centers
```

Report của cùng fiscal year xuất hiện sau cutoff:

```
KHÔNG refit centers
↓
transform bằng frozen preprocessing/model
↓
assign vào frozen centers
```

B-PIT giữ cùng feature space, thuật toán, k và scaling methodology của Financial model đã chọn; thay đổi nhằm loại timing leakage ở cấp cluster centers.

Thứ tự cluster dùng cho Financial allowed rule (Mục 3.4.1) cũng được tính tại cutoff, trên đúng tập fit, rồi lưu trong artifact B-PIT cùng centers:

```
cluster_order(t)
= sắp xếp cluster theo median(ROE) giảm dần, tie: median(debt_to_equity) tăng dần
= tính trên tập report dùng để fit tại cutoff(t)
→ đóng băng cùng centers
→ runtime chỉ đọc, không tính lại trên toàn bộ ticker của fiscal year
```

B-PIT hiện là **ĐÃ KHÓA — CHỜ BUILD + AUDIT + PIN**. Official Combined/Financial-only backtest phụ thuộc Financial clustering bị **BLOCKED** cho tới khi B-PIT artifact/version/hash và PIT audit PASS.

Financial runtime PIT ở mỗi decision_date vẫn là:

```
usable_from_date <= decision_date
↓
lọc fiscal_year đã usable
↓
selected_fiscal_year = max(fiscal_year)
```

B-PIT quyết định cluster state được xây như thế nào; runtime PIT quyết định fiscal year nào được phép sử dụng tại decision_date.

3.2. Quy tắc về độ cũ của dữ liệu tài chính (Financial Staleness — Q10)

Canonical parameter:

```
stale_months = 18
```

Financial state của fiscal year t hết hiệu lực khi xảy ra điều kiện nào đến trước:

1. fiscal year t+1 đã trở nên usable tại `decision_date`; hoặc
2. Financial state của FY t đã vượt hard cap 18 tháng kể từ `period_end`.

Nếu vượt hard cap:

```
FINANCIAL_STALE
→ Financial ineligible
```

Official runtime phải đọc `stale_months = 18` từ config đã khóa; không giữ một giá trị hard-code khác config trong implementation.

3.3. Quy tắc thanh khoản / ILLIQUID — Q2

Một mã được upstream gắn trạng thái `ILLIQUID` theo liquidity rule hiện hành khi trong cửa sổ 63 phiên có hơn 50% phiên không khớp lệnh hoặc giá trị giao dịch dưới 10 triệu đồng/ngày.

Decision Q2 đã khóa:

```
ILLIQUID
→ loại khỏi traded universe của M3
→ reason = MARKET_ILLIQUID
```

Do traded universe là universe cơ sở của các strategy, `ILLIQUID` không tham gia selection của:

- Combined
- Market-only
- Financial-only
- Equal-weight
- Momentum-only

Cần phân biệt:

```
ILLIQUID
≠
không có cluster label
```

Ticker có thể vẫn có label kỹ thuật từ upstream nhưng không đủ điều kiện tham gia M3 traded universe.

Market Eligible Pool và traded universe vẫn là hai khái niệm khác nhau: Market Eligible còn áp thêm Market cluster rule; traded universe là universe cơ sở đã loại `ILLIQUID`.

3.4. Quy tắc eligibility theo từng strategy

Các strategy M3 không bắt buộc sử dụng cùng một eligibility universe. Mỗi strategy dùng đúng universe của strategy contract.

3.4.1. Allowed-cluster rule chính thức — Q5

Market branch

Market chỉ chấp nhận leading cluster tại snapshot tương ứng.

Selector:

- primary: `median(mom_126)` cao nhất
- nếu hòa: `median(mdd_126)` cao hơn → drawdown nông hơn

Không suy leading cluster từ numeric label. Nếu selector không xác định được duy nhất theo profile/tie rule đã khóa:

```
CLUSTER_PROFILE_AMBIGUOUS
→ fail closed
```

Financial branch
Financial clusters được order theo:

- primary: `median(ROE)` cao hơn
- nếu hòa: `median(debt_to_equity)` thấp hơn

Median được tính trên tập fit của B-PIT tại `cutoff(t)` và đóng băng trong artifact B-PIT (Mục 3.1.5). Không tính lại profile tại decision_date bằng toàn bộ ticker của fiscal year, vì tập đó có thể gồm báo cáo chưa usable tại decision_date.

Canonical Q5 dùng Va:

```
Market
→ chỉ leading cluster

Financial
→ chấp nhận mọi cluster TRỪ weakest cluster
```

Tức:

```
market_rule = leading
financial_rule = not_weakest
```

Không còn dùng Market strongest ∩ Financial strongest làm canonical Combined rule.

3.4.2. Combined

```
Market Eligible Pool
=
traded universe
∩ Market leading cluster

Financial Eligible Pool
=
Financial PIT-safe state
∩ not stale
∩ B-PIT supported
```

```
∩ Financial cluster != weakest cluster
∩ Financial data-quality/coverage requirements
```

Candidate Universe:

```
Combined Candidate Universe
=
Market Eligible Pool
∩
Financial Eligible Pool
```

Candidate Universe được tạo trước khi tính MC/FC ranking.

3.4.3. Market-only

Market-only sử dụng: `Market Eligible Pool` và không yêu cầu Financial B-PIT label, Financial PIT state, Financial staleness, Financial sector coverage hoặc Financial indicators.

Luồng:

```
traded universe
↓
Market leading cluster
↓
Market ranking
↓
Top 30%
↓
Equal weight
```

3.4.4. Financial-only

Financial-only sử dụng:

```
Financial Eligible Pool
∩
traded universe
```

Không yêu cầu ticker nằm trong Market leading cluster và không dùng MC1–MC3 để ranking.

Luồng:

```
traded universe
+
Financial PIT-safe/B-PIT state
↓
Financial not-weakest eligibility
↓
Financial ranking
↓
Top 30%
```

```
↓
Equal weight
```

3.4.5. Equal-weight

Equal-weight sử dụng toàn bộ: `traded universe`.

Không dùng Market/Financial cluster eligibility, Financial PIT hoặc ranking; không chọn top 30%.

3.4.6. Momentum-only

Momentum-only sử dụng:

```
traded universe
∩ mom_126 hợp lệ
```

Rank `mom_126` giảm dần, chọn top 30%, equal weight. Không dùng Financial branch.

3.4.7. Nguyên tắc sector coverage — Q4

Do Financial B/B-PIT hiện không có coverage phù hợp cho:

- Bank
- Securities
- Insurance

Xử lý canonical:

```
Combined
→ loại nếu không có Financial B-PIT state
→ reason = FINANCIAL_SECTOR_NO_B

Financial-only
→ không thể sử dụng nếu Financial state bắt buộc không tồn tại

Market-only
→ không bị loại chỉ vì thiếu Financial B-PIT

Equal-weight
→ không bị loại chỉ vì thiếu Financial B-PIT

Momentum-only
→ không bị loại chỉ vì thiếu Financial B-PIT
```

Lý do loại trừ `FINANCIAL_SECTOR_NO_B` (và mọi lý do loại khác) được ghi vào `universe_audit_<strategy>.csv` (Mục 16.10). Báo cáo M3 phải nêu tỷ lệ ticker bị loại bởi `FINANCIAL_SECTOR_NO_B` theo kỳ cho Combined, tính trực tiếp từ file này.

Ghi chú implementation: `strategies.py` đã tạo audit này trong bộ nhớ nhưng `run_m3.py` hiện chưa ghi ra đĩa; đây là việc sync code ở Giai đoạn 2.

3.4.8. Nguyên tắc tổng quát

```
decision_date
      │
      ├── Combined
      │   → Market Eligible ∩ Financial Eligible
      │
      ├── Market-only
      │   → Market Eligible Pool
      │
      ├── Financial-only
      │   → Financial Eligible Pool ∩ traded universe
      │
      ├── Equal-weight
      │   → traded universe
      │
      └── Momentum-only
          → traded universe ∩ mom_126 available
```

Financial coverage không được dùng để thu hẹp Market-only, Equal-weight hoặc Momentum-only.

3.5. Sáu tiêu chí chọn mã và xếp hạng kết hợp

Q7–Q9 đã được owner khóa. Các criterion dưới đây là official methodology.

3.5.1. Ba tiêu chí thị trường — Q7

```
MC1 = mom_126
→ Động lượng giá khoảng 126 phiên, higher is better

MC2 = downside_vol_63
→ Biến động giảm 63 phiên, lower is better

MC3 = liquidity_21
→ Thanh khoản / giá trị giao dịch 21 phiên (dùng giá trị thô, xếp hạng bằng hàm phân vị percer
```

Các giá trị được lấy từ Market snapshot tại `decision_date`.

3.5.2. Ba tiêu chí tài chính — Q8

```
FC1 = ROE
→ Khả năng sinh lời trên vốn chủ sở hữu, higher is better

FC2 = F-score
→ Điểm sức khỏe tài chính 0–9, higher is better

FC3 = Z-score
→ Chỉ báo an toàn/rủi ro tài chính, higher is better
```

Financial criteria của ticker được lấy theo:

```
selected_fiscal_year
```

đã xác định bằng Financial PIT.

3.5.3. Missing Financial criterion — Q8

Nếu thiếu một hoặc một phần FC1–FC3:

```
missing criterion
→ percentile trung tính = 0.5
→ flag = FC_MISSING
```

Không biến missing thành 0.

Nếu cả ba đều thiếu:

```
ROE missing
AND F-score missing
AND Z-score missing
→ FINANCIAL_CRITERIA_MISSING
→ loại khỏi Financial scoring
```

Ticker này không thể đi tiếp ở strategy cần Financial scoring tại kỳ đó.

`FINANCIAL_CRITERIA_MISSING` là một lý do loại khỏi Candidate Universe của strategy cần Financial scoring, giống các lý do eligibility khác:

```
Candidate Universe
= các ticker đã qua eligibility
  VÀ không bị FINANCIAL_CRITERIA_MISSING (với strategy cần Financial scoring)

N_candidates
= số ticker của Candidate Universe SAU bước loại này
```

`N_candidates` này được dùng chung cho kiểm tra `N_min` (Mục 3.7) và cho `n_select = ceil(0.30 × N_candidates)` (Mục 3.6.1).

Cờ `FC_MISSING` và lý do `FINANCIAL_CRITERIA_MISSING` được ghi vào `universe_audit_<strategy>.csv` (Mục 16.10).

Ghi chú implementation: code hiện kiểm tra `N_min` trên số ticker TRƯỚC khi loại `FINANCIAL_CRITERIA_MISSING` và loại ticker này mà không ghi lý do; cần sync ở Giai đoạn 2.

3.5.4. Hai kết quả xếp hạng riêng

Market score:

```
S_market
```

```
=
mean(
  percentile(MC1),
  percentile(MC2),
  percentile(MC3)
)
```

Financial score:

```
S_fin
=
mean(
  percentile(FC1),
  percentile(FC2),
  percentile(FC3)
)
```

sau khi áp missing rule 0.5.

One-sided strategy:

```
Market-only    → score = S_market
Financial-only → score = S_fin
```

3.5.5. Quy tắc xếp hạng Combined — Q9

```
Combined Score
=
0.5 × S_market
+
0.5 × S_fin
```

Tie-break:

```
Combined Score cao hơn
↓
liquidity_21 cao hơn
↓
ticker A → Z
```

Sau ranking:

```
n_select = ceil(0.30 × N_candidates)
```

Chọn đúng `n_select` ; không mở rộng số lượng chỉ vì score ở cutoff bằng nhau. Các ticker được chọn nhận equal weight theo Mục 3.6.

3.6. Quy tắc selection và phân bổ tỷ trọng

Đã được chủ dự án duyệt.

3.6.1. Quy tắc chọn top 30%

Rule này áp dụng cho các strategy có `top-selection`:

```
Combined
Market-only
Financial-only
Momentum-only
```

Sau khi strategy đã tạo Candidate Universe và ranking theo rule riêng:

```
Candidate Universe của strategy
↓
ranking của strategy
↓
n_select = ceil(0.30 × N_candidates)
↓
chọn top n_select
↓
Equal weight
```

Không áp dụng top 30% cho:

```
Equal-weight
```

vì Equal-weight sử dụng toàn bộ universe hợp lệ của chính strategy.

3.6.2. Phân bổ trọng số Equal Weight

Sau khi strategy đã xác định danh sách ticker cần nắm:

```
N_selected
=
số ticker được chọn
```

mỗi ticker có:

```
target_weight
=
1 / N_selected
```

Trong đó:

```
Combined / Market-only / Financial-only / Momentum-only
→ N_selected = n_select
```

```
Equal-weight
→ N_selected = toàn bộ số ticker hợp lệ trong traded universe
```

3.7. Quy tắc ngưỡng quy mô danh mục tối thiểu (`UNIVERSE_TOO_SMALL` — Q6)

Q6 đã khóa:

```
n_min = 10
```

Áp dụng cho:

- Combined
- Market-only
- Financial-only

Không áp dụng cho:

- Equal-weight
- Momentum-only

`N_candidates` ở mục này là số ticker của Candidate Universe sau khi đã loại `FINANCIAL_CRITERIA_MISSING` (Mục 3.5.3), tức đúng số ticker được đưa vào ranking.

3.7.1. Khi Candidate Universe quá nhỏ

Nếu:

```
N_candidates < 10
```

thì strategy không mở vị thế mới.

```
mã đang nắm + vẫn còn trong Candidate Universe
→ HOLD

mã đang nắm + đã rời Candidate Universe
→ SELL

candidate mới + chưa nắm giữ
→ không BUY
```

Tiền thu được từ SELL thành công giữ ở CASH, không phân bổ cho candidate mới.

3.7.2. Trường hợp kỳ đầu tiên

Nếu ngay kỳ đầu:

```
N_candidates < 10
```

thì giữ:

```
100% CASH
```

cho tới kỳ đầu tiên `N_candidates >= 10`.

Status audit:

```
UNIVERSE_TOO_SMALL
```

Lý do: tránh danh mục bị chi phối bởi một số lượng ticker quá nhỏ và duy trì minimum diversification gate đã khóa.

3.8. Quy tắc khớp lệnh và xử lý giao dịch thất bại

Các quy tắc sau điều khiển cách thức danh mục thực thi chuyển đổi từ Current Holdings sang Target Portfolio:

3.8.1. Lịch thực thi

Giao dịch được thực hiện ở giá đóng cửa của phiên hợp lệ kế tiếp:

```
execution_date = phiên giao dịch hợp lệ đầu tiên sau decision_date (T+1)
```

3.8.2. Thứ tự thực thi lệnh

- **SELL luôn được thực hiện trước**: Nhằm giải phóng sức mua và xác định số dư Cash thực tế.
- **BUY được thực hiện sau**: Dựa trên lượng tiền mặt thực tế thu được sau khi hoàn tất các lệnh SELL.

3.8.3. Xử lý lệnh mua thất bại (BUY_FAILED)

Phần vốn dự kiến mua mã đó tiếp tục giữ ở Cash; không chọn mã khác thay thế và không chia lại phần vốn của chính `BUY_FAILED` cho mã khác.

3.8.4. Xử lý lệnh bán thất bại (SELL_FAILED)

Mã vẫn nằm trong holdings và không tạo Cash.

3.8.5. Cơ chế bù trừ vốn tự động (Proportional Scaling khi thiếu Cash do SELL_FAILED)

Nếu vì `SELL_FAILED` mà Cash thực có không đủ cho các lệnh BUY còn lại:

- Áp dụng **proportional scaling** cho toàn bộ các BUY còn lại theo tỷ lệ:

```
execution_scale = available_cash / total_required_buy_cash
```

- Không dùng margin, không thay mã, không đổi ranking.

3.8.6. Tradability / ceiling-floor — Q12

Tại `execution_date`, một lệnh chỉ executable khi có execution price hợp lệ, volume > 0 và không vi phạm ceiling/floor rule.

```
volume = 0
→ BUY_FAILED
→ SELL_FAILED

raw_close = ceiling
→ BUY_FAILED

raw_close = floor
→ SELL_FAILED
```

Không có đủ order-book/dư mua-dư bán nên M3 không giả định lệnh vẫn khớp khi đóng cửa ở giá trần/sàn.

Ceiling/floor được kiểm tra trên raw market price của đúng phiên; portfolio valuation/return basis vẫn theo Q11.

Nếu không có dữ liệu ceiling/floor/raw_close cho ticker tại `execution_date`, lệnh **không** được coi là executable (fail-closed):

```
ceiling/floor data missing
→ BUY_FAILED hoặc SELL_FAILED
→ flag = LIMIT_DATA_MISSING
→ đếm trong metrics.json (limit_data_missing)
```

Ghi chú implementation: `PriceBook.tradable` hiện trả `True` khi thiếu dòng trần/sàn; cần sync ở Giai đoạn 2. Readiness audit (Giai đoạn 1) phải báo số cặp (ticker, execution_date) thiếu dữ liệu trần/sàn trước official run.

3.9. Quy tắc kết thúc segment/run

Quy tắc này xác định cách M3 xử lý điểm cuối của một segment/run và tránh sử dụng execution, valuation hoặc dữ liệu giá ngoài phạm vi segment.

A. Target tại boundary

`segment_end_date` là **ngày biên kết thúc theo lịch của một segment backtest**, dùng để giới hạn phạm vi thời gian mà segment đó được phép sử dụng cho execution, valuation và dữ liệu giá.

`segment_end_date` không phải là decision_date cuối cùng; decision cuối cùng có thể xảy ra trước ngày này.

Cần phân biệt:

```
last_decision_date
```

```
= decision_date cuối cùng của segment
= ngày cuối cùng strategy được phép tạo Target Portfolio

segment_end_date
= ngày biên kết thúc của toàn segment
= giới hạn thời gian tối đa mà execution, valuation
  và dữ liệu giá được phép thuộc về segment đó
```

Do đó:

```
last_decision_date
không mặc định bằng
segment_end_date
```

Decision snapshot cuối cùng có thể xảy ra trước `segment_end_date`.

Ví dụ, nếu một segment có:

```
last_decision_date = 29/12
segment_end_date   = 31/12
```

thì Target Portfolio vẫn được tạo tại decision ngày 29/12. Tuy nhiên, nếu phiên giao dịch hợp lệ kế tiếp để thực hiện Target đó nằm sau ngày 31/12 thì execution đã vượt ra ngoài segment và không được phép thực hiện.

Về mặt runtime, nếu:

```
decision_date <= segment_end_date
```

thì Bước 1 vẫn được chạy đầy đủ:

```
Candidate Universe
→ score/ranking
→ Target Portfolio
→ lưu targets_<strategy>.csv
```

Việc một Target được tạo trong segment không có nghĩa Target đó chắc chắn được thực thi trong cùng segment.

Sau khi Target được tạo, engine phải kiểm tra `execution_date`.

Nếu:

```
execution_date > segment_end_date
```

thì Target vẫn được lưu để phục vụ audit và reproducibility, nhưng không được thực thi:

```
status = NOT_EXECUTED_SEGMENT_END
```

```
→ không đưa vào execution schedule
→ không chạy Bước 2
→ không BUY
→ không SELL
→ không tạo turnover
→ không thay đổi Actual Holdings
→ không sử dụng price nằm sau segment_end_date
```

Quy tắc này nhằm bảo đảm một decision được tạo sát cuối segment không khiến backtest sử dụng execution price hoặc dữ liệu giá thuộc segment kế tiếp.

B. Segment/run hữu hạn

Một segment hữu hạn phải có `segment_end_date` xác định trước.

```
end_date
= phiên giao dịch hợp lệ cuối cùng
  không vượt segment_end_date
```

Actual Holdings đang tồn tại được giữ tới `end_date`, revalue tại `end_date`, dùng `PRICE_STALE` nếu thiếu giá và chốt NAV cuối segment.

Tại `end_date`:

- không mở rebalance mới
- không BUY
- không SELL
- không sử dụng dữ liệu giá sau `segment_end_date`

Giá trị canonical và hệ quả

```
segment_end_dates
development = 2023-12-29
holdout     = 2025-12-31
```

Hệ quả phải ghi trong báo cáo:

```
Development
→ last_decision_date = segment_end_date = 2023-12-29
→ execution_date của decision 29/12/2023 là 02/01/2024 > segment_end_date
→ decision này luôn NOT_EXECUTED_SEGMENT_END
→ end_date = 2023-12-29; NAV cuối Development chốt tại phiên này

Khoảng 30/12/2023 → 01/02/2024
→ không thuộc NAV của segment nào
→ Holdout bắt đầu từ 100% CASH và giao dịch lần đầu ngày 01/02/2024
```

Đây là chủ ý của strict isolation, không phải lỗi dữ liệu. Không nối NAV Development và Holdout thành một chuỗi liên tục khi báo cáo.

C. Rolling/live update chưa có ngày kết thúc

Nếu chưa có `segment_end_date`, snapshot mới nhất hiện tại không tự động được coi là snapshot cuối. Target vẫn được tạo/lưu và chờ execution theo chu trình bình thường.

---

4. CÁC QUYẾT ĐỊNH ĐÃ KHÓA VÀ DEPENDENCY CÒN MỞ

Hạng mục	Trạng thái
Chỉ mua, không bán khống	ĐÃ KHÓA
Không dùng đòn bẩy	ĐÃ KHÓA
Thị trường và tài chính phân cụm riêng	ĐÃ KHÓA
Kết hợp ở tầng strategy/portfolio, không phân cụm lại đặc trưng thô	ĐÃ KHÓA
Q1: Market model = A v4, k=3; M2 K=2 giữ legacy/research control	ĐÃ KHÓA
Q2: ILLIQUID loại khỏi traded universe	ĐÃ KHÓA
Q3: Financial clustering chính thức = B-PIT	ĐÃ KHÓA — CHỜ BUILD/AUDIT/PIN
Q3 làm rõ: cutoff B-PIT = max(30/04/(t+1), ngày usable theo Q-date); cluster order đóng băng tại cutoff	CẦN OWNER XÁC NHẬN (R1, R2)
Q4: Bank/CK/Bảo hiểm không vào Combined nếu không có Financial B-PIT state; vẫn có thể vào Market-only/Equal-weight/Momentum-only	ĐÃ KHÓA
Q5: Market = leading cluster; Financial = not-weakest; Combined dùng Va	ĐÃ KHÓA
Q6: N_min = 10 cho Combined/Market-only/Financial-only	ĐÃ KHÓA
Q7: MC1 = mom_126; MC2 = downside_vol_63; MC3 = liquidity_21	ĐÃ KHÓA
Q8: FC1 = ROE; FC2 = F-score; FC3 = Z-score; partial missing → 0.5 + flag; all missing → loại	ĐÃ KHÓA
Q9: 0.5 × S_market + 0.5 × S_fin; tie → liquidity cao hơn → ticker A→Z	ĐÃ KHÓA
Financial PIT runtime: usable_from_date <= decision_date rồi chọn max(fiscal_year)	ĐÃ KHÓA
Q-date: FY>=2022 dùng CafeF disclosure; FY<2022 dùng lag sau 30/06 t+1; 30/04 sensitivity	ĐÃ KHÓA
Q10: Financial staleness = FY kế tiếp usable hoặc hard cap 18 tháng sau period_end	ĐÃ KHÓA

Chọn top 30%, làm tròn lên, equal weight	ĐÃ KHÓA
Tái cân bằng hàng tháng	ĐÃ KHÓA
Giao dịch ở giá đóng cửa phiên hợp lệ kế tiếp	ĐÃ KHÓA
Q12: volume 0 chặn hai chiều; close=ceiling chặn BUY; close=floor chặn SELL	ĐÃ KHÓA
BUY_FAILED giữ vốn ở Cash; không thay mã/không chia lại	ĐÃ KHÓA
SELL_FAILED giữ position và không tạo Cash	ĐÃ KHÓA
SELL_FAILED gây thiếu Cash → proportional scaling toàn bộ BUY còn lại	ĐÃ KHÓA
Segment boundary: Target vẫn tạo/lưu; execution_date > segment_end → NOT_EXECUTED_SEGMENT_END, không execution; Development segment_end = 2023-12-29	ĐÃ KHÓA — CODE PHẢI SYNC
Missing valuation price → PRICE_STALE stale mark semantics	ĐÃ KHÓA
DELISTED_EXIT	PHẦN MỞ RỘNG — CHƯA IMPLEMENT
Q11 main return basis = adj_close adjusted-return proxy	ĐÃ KHÓA
Secondary price-only	NOT_READY — CHƯA CÓ CORPORATE-ACTION READINESS — KHÔNG BLOCK MAIN
Lợi suất tiền mặt = 0	ĐÃ KHÓA
Main transaction cost = 0	ĐÃ KHÓA
Main slippage = 0	ĐÃ KHÓA
Cost sensitivity 15/30 bps	PHẦN MỞ RỘNG
Q13 one-way turnover = (executed BUY + executed SELL)/(2 × pre_trade_NAV)	ĐÃ KHÓA — CHỜ GIT SYNC NẾU CẦN
Q14 benchmark = VNINDEX; controls = Market-only, Financial-only, Equal-weight, Momentum-only	ĐÃ KHÓA
M2 K=2: chỉ báo cáo số ứng viên theo kỳ làm bằng chứng so sánh, không chạy backtest	CẦN OWNER XÁC NHẬN (R7)
Thiếu dữ liệu trần/sàn → fail-closed, flag LIMIT_DATA_MISSING	ĐỀ XUẤT LÀM RÕ Q12 (R5)
Q15 Holdout sạch về portfolio-rule/return-based evaluation nhưng không tuyên bố hoàn toàn unseen về upstream model selection	ĐÃ KHÓA
Q16 Final bắt đầu từ snapshot cuối 01/2026; exact end khóa sau	ĐÃ KHÓA NGUYÊN TẮC — EXACT

readiness audit trước khi xem performance	END CHỜ READINESS
Q17 adjusted-price one-session glitch → derived repair + flag, source immutable	ĐÃ KHÓA / IMPLEMENTED — GIỮ TEST
Historical schedule = Development 2019–2023, Holdout 2024–2025	ĐÃ KHÓA
Sharpe chỉ báo cáo ở Final Evaluation	ĐÃ KHÓA
Canonical outputs = targets/ledger/holdings/turnover/nav/benchmark/metrics/manifest	ĐÃ KHÓA
Bổ sung canonical output `universe_audit_<strategy>.csv`	ĐỀ XUẤT (R6)

Các mục CHỜ còn lại sau bảng này chỉ được phép là readiness/implementation dependency, không phải methodology decision chưa chốt.

---

5. KHUNG DỮ LIỆU CHUNG

Mục này là nơi duy nhất trong tài liệu mô tả chi tiết **runtime input contract của M3-A**.

Các mục khác chỉ dẫn chiếu về Mục 5, không lặp lại toàn bộ path/schema.

5.1. Đầu vào từ hệ Market

M3-A sử dụng hai nhóm input Market/execution.

A. `terminal.sqlite`

```
data/web/delta-intelligence-v4/terminal.sqlite
```

Cung cấp:

```
Market observations → các snapshot thị trường theo từng decision_date và ticker
Financial observations → các quan sát tài chính theo từng fiscal_year và ticker
price table → bảng giá giao dịch theo ngày của từng ticker
```

Đối với Market observations, M3 cần các thông tin tương ứng với:

```
date
ticker
cluster
features
liquidity
```

Các Market criteria cần cho strategy như:

```
mom_126
downside_vol_63
liquidity_21
mdd_126
```

được đọc từ Market state đã được upstream tạo.

M3-A không fit lại Market clustering trong quá trình backtest.

Price data trong `terminal.sqlite` cung cấp tối thiểu:

```
ticker
date
adj_close
raw_close
volume
```

Vai trò cụ thể của raw/adjusted price đối với return phải tuân theo return-basis decision đã khóa.

Adjusted-price one-session glitch handling — Q17

Trước khi `adj_close` được sử dụng trong `PriceBook` /runtime, M3 áp dụng detector/repair Q17 cho lỗi giá điều chỉnh lệch đúng một phiên.

Với mỗi ticker, xét tỷ lệ:

```
adj_close / raw_close
```

của ba phiên liên tiếp: phiên trước, phiên hiện tại và phiên kế tiếp.

Phiên hiện tại được coi là **one-session adjusted-price glitch** khi đồng thời thỏa:

```
| ratio_current / ratio_previous - 1 | > 3%
```

và:

```
| ratio_next / ratio_previous - 1 | < 0.5%
```

Trong đó:

```
ratio
=
adj_close / raw_close
```

Nếu thỏa điều kiện trên, `adj_close` của phiên hiện tại không được dùng trực tiếp. Thay vào đó, runtime tạo giá điều chỉnh đã repair:

```
repaired_adj_close
=
```

```
raw_close_current
×
(adj_close_next / raw_close_next)
```

và gắn cờ:

```
ADJ_GLITCH_REPAIRED
```

Nguyên tắc xử lý:

```
source artifact
→ không sửa

raw_close nguồn
→ không sửa

adj_close nguồn
→ không overwrite

repair
→ chỉ tạo derived/runtime adjusted price
```

`PriceBook` sử dụng `repaired_adj_close` thay cho `adj_close` gốc tại đúng phiên bị phát hiện glitch.

Detector/repair phải deterministic và auditable; cùng một input và cùng version code phải tạo cùng kết quả repair.

Q17 không được dùng để tùy ý làm mượt giá hoặc sửa các biến động lớn thông thường. Chỉ những phiên thỏa chính xác detector nêu trên mới được gắn:

```
ADJ_GLITCH_REPAIRED
```

B. C8 canonical Market artifacts

```
artifacts/cafef_primary/cafef-c8-complete-only-v2/canonical/
├── prices_daily.jsonl
├── trading_calendar.jsonl
└── benchmark_daily.jsonl
```

Vai trò:

```
prices_daily.jsonl
→ raw market information
→ ceiling/floor information
→ tradability support

trading_calendar.jsonl
→ trading calendar
→ xác định phiên hợp lệ kế tiếp
```

```
benchmark_daily.jsonl
→ VNINDEX benchmark
```

Các file này không được tự dùng thay thế lẫn nhau nếu semantic khác nhau.

5.2. Đầu vào từ hệ Financial

M3-A sử dụng ba thành phần Financial chính.

A. Financial B-PIT state

Official Combined/Financial-only backtest sử dụng Financial clustering state từ B-PIT đã build/audit/version/hash/pin.

Current B v4 state có thể làm source/reference để xây B-PIT nhưng không đủ để chạy official Financial-dependent backtest nếu B-PIT chưa READY.

B-PIT phải bảo đảm:

```
cutoff FY t = max(30/04/(t+1), ngày usable theo Q-date)   (Mục 3.1.5)
chỉ các báo cáo đã đủ điều kiện sử dụng tại cutoff
→ được dùng để khớp mô hình và xác định tâm cụm cố định
→ cùng tập này dùng để tính và đóng băng cluster_order

các báo cáo của cùng FY xuất hiện sau cutoff
→ chỉ được gán vào các tâm cụm đã đóng băng

báo cáo xuất hiện trong tương lai
→ không được dùng để khớp lại hoặc làm thay đổi
   các tâm cụm lịch sử
```

Artifact B-PIT tối thiểu phải chứa, theo từng fiscal year:

```
fiscal_year
cutoff_date
n_fit (số report dùng để fit)
n_assigned_after_cutoff
preprocessing params + centers
cluster_order (best → worst)
labels: ticker, fiscal_year, cluster, fitted_or_assigned
sha256
```

Audit bắt buộc: không report nào có `usable_from_date > cutoff_date` nằm trong tập fit; báo cáo ARI giữa B-PIT và B v4 theo từng năm.

Trạng thái hiện tại:

```
B-PIT = NOT_READY
```

cho tới khi build + PIT audit + pin PASS.

M3-A không refit B-PIT trong backtest; nó đọc upstream B-PIT state đã pin và sau đó áp Financial runtime PIT tại `decision_date`.

B. Financial indicators

```
data/financial/indicators_v1/results/
indicators-2c4a24e3830d325f/indicators.sqlite
```

M3 hiện sử dụng Financial criteria:

```
ROE
F-score
Z-score
```

và metadata:

```
sector_group
```

khi strategy/eligibility rule yêu cầu.

Theo Q8 đã khóa:

```
missing một hoặc một phần Financial criteria
→ percentile = 0.5
→ missing flag

missing toàn bộ Financial criteria
→ loại khỏi Financial scoring
```

C. Financial availability

```
data/financial/disclosure_dates_v1/
availability-4f0f83108e761dae/availability.csv
```

Tối thiểu cần:

```
ticker
fiscal_year
usable_from_date
```

Tại:

```
decision_date = t
```

các fiscal year hợp lệ là:

```
usable_from_date <= decision_date
```

Sau đó:

```
selected_fiscal_year
=
max(usable_financial_years)
```

Nếu không có fiscal year hợp lệ:

```
FINANCIAL_NOT_AVAILABLE
```

Không dùng `period_end` để thay thế thời điểm dữ liệu được phép sử dụng.

5.3. Các phép xử lý bắt buộc sau khi đọc input

M3-A vẫn phải thực hiện:

```
Market state
        +
Financial state PIT-safe
        ↓
Eligibility
        ↓
Strategy-specific Candidate Universe
        ↓
Strategy-specific ranking
        ↓
Target Portfolio
```

Không bắt buộc persist riêng toàn bộ bảng intermediate nếu canonical implementation không yêu cầu.

Kết quả persist của portfolio-selection stage là:

```
universe_audit_<strategy>.csv   (eligibility + lý do loại, Mục 16.10)
targets_<strategy>.csv
```

Các phép PIT/eligibility/ranking phải có thể tái lập từ:

```
manifest.json
+
config đã hash
+
version/hash của runtime inputs
+
code commit
```

5.4. Historical backtest segments

Historical M3-A sử dụng:

Segment	Khoảng thời gian	Decision snapshots	Vai trò
Development	2019-01-31 → 2023-12-29	60	Historical development evaluation
Holdout	2024-01-31 → 2025-12-31	24	Chạy bằng cùng frozen strategy/config

Tổng:

```
84 decision snapshots
```

Development và Holdout phải được chạy/seal riêng.

Không dùng kết quả Development để thay strategy trước khi mở Holdout.

Dữ liệu sau:

```
2025-12-31
```

thuộc vùng ứng viên Final Evaluation.

Exact Final Evaluation window chỉ được khóa sau khi data readiness và evaluation-status audit đạt yêu cầu.

PHẦN A — M3-A: BACKTEST THEO 4 BƯỚC, BẮT ĐẦU TỪ CÁC M3 RUNTIME INPUT ĐÃ PIN

6. ĐIỂM BẮT ĐẦU CỦA M3-A

Mục này không định nghĩa lại input contract.

Chi tiết sáu logical runtime inputs được giữ duy nhất tại Mục 5 — Khung dữ liệu chung.

M3-A chỉ bắt đầu khi toàn bộ runtime inputs bắt buộc đã:

```
đặt đúng logical path/version
        +
pass schema audit
        +
pass coverage audit cho segment cần chạy
        +
version/hash được xác minh
        +
```

```
được pin vào manifest
```

Nếu một runtime input bắt buộc chưa đạt:

```
INPUT_NOT_READY
```

thì official backtest phụ thuộc input đó không được chạy.

M3-A không chạy lại:

```
Market A clustering
Financial B-PIT clustering
```

trong backtest.

Hai hệ clustering là upstream state của M3-A.

M2 K=2 cũ chỉ được dùng khi một research/control strategy hoặc config đã khóa yêu cầu; nó không phải runtime prerequisite của main M3-A.

Sau khi input readiness PASS:

```
Runtime inputs ready
        ↓
Strategy rules/config ready
        ↓
Target Portfolio
        ↓
Execution
        ↓
Holding / valuation
        ↓
Rebalance
        ↓
Canonical outputs
```

Nếu config còn:

```
status = PROPOSED
```

thì chỉ được thực hiện các bước readiness/targets-only được protocol cho phép.

Official backtest có return chỉ chạy sau khi:

```
config = LOCKED
```

---

7. LUỒNG HOẠT ĐỘNG DUY NHẤT CỦA BACKTEST

Từ lúc sáu GitHub M3 runtime inputs đã audit/pin đầy đủ, toàn bộ M3-A được tổ chức theo đúng 4 bước:

```
ĐIỂM BẮT ĐẦU
Có đủ GitHub M3 runtime inputs đã audit/pin
        ↓
1. CHỌN CỔ PHIẾU
        ↓
2. GIẢ LẬP MUA/BÁN
        ↓
3. GIỮ VÀ TÍNH LỜI/LỖ
        ↓
4. THÁNG SAU TÁI CÂN BẰNG
        ↓
   quay lại Bước 1
```

Bốn bước này được lặp lại cho đến hết Development hoặc Holdout theo 84 Market snapshots hiện hành trong `terminal.sqlite`, với các input phụ trợ đã pin cùng run.

Sau khi đã chạy xong 4 bước trên cho toàn bộ lịch sử mới tính các performance metrics và so sánh benchmark.

BẢNG CÁC BIẾN TÍNH TOÁN TRONG 4 BƯỚC M3-A

Ghi chú:

- Các thuật ngữ cốt lõi như `decision_date`, `execution_date`, `ILLIQUID`, `traded universe`, `Market Eligible Pool`, `Financial Eligible Pool`, `Candidate Universe`, `Target Portfolio` và `Current Portfolio` đã được định nghĩa tại **Bảng thuật ngữ và biến cốt lõi M3-A**.
- Bảng này chỉ giữ các biến phục vụ trực tiếp cho tính toán, state của engine hoặc canonical output trong 4 bước.
- `values[ticker]` là state thực tế trong engine, lưu giá trị phần vốn đang nắm giữ ở ticker.
- `marks[ticker]` là giá gần nhất được engine dùng để định giá ticker.
- `holdings_<strategy>.csv` không persist `values[ticker]`; output holdings theo Git hiện hành chỉ ghi `date`, `ticker`, `weight`.

Bước	Biến / trường	Ý nghĩa	Đơn vị / dạng
1	`usable_financial_years`	Tập các năm tài chính thỏa usable_from_date <= decision_date.	Tập năm
1	`MC1 / mom_126`	Động lượng 6 tháng; higher is better.	Feature / percentile
1	`MC2 / downside_vol_63`	Biến động giảm 63 phiên; lower is better.	Feature / percentile
			Feature /

1	MC3 / liquidity_21	Thanh khoản/GTGD 21 phiên; higher is better.	percentile
1	FC1 / ROE	Return on Equity; higher is better.	Indicator / percentile
1	FC2 / F-score	Piotroski F-score 0–9; higher is better.	Indicator / percentile
1	FC3 / Z-score	Chỉ báo an toàn/rủi ro tài chính; higher is better.	Indicator / percentile
1	S_market	Trung bình percentile của MC1–MC3.	0–1
1	S_fin	Trung bình percentile của FC1–FC3 sau missing rule.	0–1
1	combined_score	0.5 × S_market + 0.5 × S_fin.	0–1
1	n_min	Minimum candidate gate = 10 cho Combined/Market-only/Financial-only.	Số nguyên
1	rank	Thứ hạng ticker theo ranking rule của strategy.	Số nguyên
1	n_select	Số mã được chọn đối với strategy có top-selection: ceil(0.30 × N_candidates).	Số nguyên
1	target_weight	Tỷ trọng mục tiêu của ticker trong Target Portfolio.	Tỷ lệ 0–1
1	status trong targets	Trạng thái Target, gồm normal/ UNIVERSE_TOO_SMALL / NOT_EXECUTED_SEGMENT_END khi phù hợp.	Chuỗi
1–4	segment_end_date	Ngày biên kết thúc theo lịch của một evaluation segment/run; được xác định độc lập với decision_date và decision snapshot cuối cùng, đồng thời là giới hạn thời gian tối đa được phép dùng cho execution, valuation và dữ liệu giá. Không mặc định bằng decision snapshot cuối cùng.	Ngày
2–4	pf / Portfolio	Object danh mục được engine giữ xuyên suốt các kỳ.	Object
2–4	cash	Giá trị tiền mặt hiện có trong Portfolio.	Value space
2–4	values[ticker]	Giá trị phần vốn hiện đang nắm giữ ở ticker trong internal Portfolio state.	Value space
2–4	marks[ticker]	Giá gần nhất được dùng để định giá ticker.	Giá
2	current_price	Giá ticker tại thời điểm revalue/execution theo price basis được sử dụng.	Giá

2	`price(ticker, date)`	Hàm trả price được sử dụng cho valuation/execution tại một ngày.	Giá hoặc None
2	`PRICE_STALE`	Trạng thái khi thiếu giá mới để revalue; engine giữ values[ticker] và marks[ticker] theo stale-mark semantics.	Cờ
2	`pre_trade_NAV / nav`	NAV sau khi revalue Current Portfolio nhưng trước rebalance.	Value space
2	`pre_weight`	Tỷ trọng ticker trước giao dịch; đối với cổ phiếu: values[ticker] / pre_trade_NAV.	Tỷ lệ 0–1
2	`delta_weight / d`	Chênh lệch giữa tỷ trọng mục tiêu và tỷ trọng trước giao dịch: target_weight - pre_weight.	Tỷ lệ
2	`side`	Hướng hành động được sinh từ delta_weight: BUY, SELL hoặc HOLD.	Chuỗi
2	`intended`	Giá trị giao dịch mà engine dự kiến thực hiện trước khi xét failed trade/scaling.	Value space
2	`available_cash`	Cash thực tế có thể sử dụng cho BUY sau khi xử lý SELL.	Value space
2	`required_buy_cash / required`	Tổng giá trị intended của các BUY có khả năng thực thi trước scaling.	Value space
2	`execution_scale / scale`	Hệ số thu nhỏ đồng đều các BUY khi Cash không đủ; tối đa bằng 1.	Tỷ lệ 0–1
2	`executed`	Giá trị giao dịch thực tế được thực hiện sau tradability check và scaling.	Value space
2	`status trong ledger`	Trạng thái execution của row, gồm EXECUTED, BUY_FAILED, SELL_FAILED, SCALED, HOLD.	Chuỗi
2	`traded`	Tổng giá trị BUY và SELL thực sự được executed tại kỳ.	Value space
2	`turnover`	One-way turnover theo contract hiện tại: traded / (2 × pre_trade_NAV).	Tỷ lệ
2	`weight`	Tỷ trọng output từ pf.weights(); cổ phiếu = values[ticker] / NAV, CASH = cash / NAV.	Tỷ lệ 0–1
3	`nav`	Giá trị toàn bộ Portfolio tại các valuation date được engine ghi nhận.	Value space
3	`stale`	Danh sách các cặp (date, ticker) gặp PRICE_STALE.	Danh sách
3	`end_date`	Ngày dùng để chốt valuation/NAV cuối segment theo	Ngày

		segment-end rule đã khóa.	
4	`schedule`	Chuỗi các kỳ (decision_date, execution_date, target) mà engine xử lý lần lượt.	Danh sách
4	`goal`	Target Portfolio thực tế được engine sử dụng cho kỳ rebalance hiện tại.	Dict
4	`Target Portfolio mới`	Target được Bước 1 tính lại tại decision_date của kỳ kế tiếp; không carry Target cũ sang kỳ mới.	State

---

8. TỔNG QUAN NHIỆM VỤ NẰM TRONG 4 BƯỚC

8.1. BƯỚC 1 — CHỌN CỔ PHIẾU

```
1.1  Xác minh dữ liệu Market
1.2  Xác minh dữ liệu Financial clustering
1.3  Căn PIT Market–Financial
1.4  Tạo Candidate Universe
1.5  Tính MC1–MC3 và FC1–FC3
1.6  Xếp hạng Combined
1.7  Chọn top 30% và tạo Target Portfolio
```

Các bước 1.1–1.6 là internal computation/validation. Output persist chính:

```
targets_<strategy>.csv
```

8.2. BƯỚC 2 — GIẢ LẬP MUA/BÁN

```
2.1  Map decision_date → execution_date
2.2  Revalue Current Portfolio tại execution_date
2.3  So Current-vs-Target
2.4  Sinh SELL trước, BUY sau, HOLD khi delta≈0
2.5  Kiểm tra price/tradability theo engine GitHub
2.6  Xử lý BUY_FAILED / SELL_FAILED / proportional scaling
2.7  Chốt ledger, holdings và turnover
```

Output persist:

```
trade_ledger_<strategy>.csv
holdings_<strategy>.csv
turnover_<strategy>.csv
```

8.3. BƯỚC 3 — GIỮ VÀ TÍNH LỜI/LỖ

Portfolio state được giữ trong engine. Tại execution date kế tiếp hoặc `end_date`, engine revalue holdings bằng price basis đã được khóa. Missing price dùng last valid mark và ghi `PRICE_STALE`.

Output persist:

```
nav_<strategy>.csv
```

Các security return/contribution và pre-rebalance-weight có thể được tính nội bộ hoặc suy từ state nhưng không còn là canonical file riêng.

8.4. BƯỚC 4 — THÁNG SAU TÁI CÂN BẰNG

Không tạo handoff CSV riêng. Portfolio state được carry trực tiếp trong engine:

```
Actual Holdings kỳ t
→ revalue tại execution_date kỳ t+1
→ Target mới
→ Current-vs-Target
→ rebalance mới
```

Sau khi chạy hết segment mới tính/report metrics và benchmark.

9. BƯỚC 1 — CHỌN CỔ PHIẾU VÀ TẠO TARGET PORTFOLIO CHO 5 STRATEGY

Mục tiêu của Bước 1 là tạo **Target Portfolio riêng cho từng strategy tại mỗi `decision_date`**, không sử dụng thông tin tương lai.

Config hiện tại khai báo 5 strategy:

```
1. combined
2. market_only
3. financial_only
4. equal_weight
5. momentum_only
```

Mỗi strategy:

```
cùng decision_date
↓
đọc đúng dữ liệu được phép sử dụng tại decision_date
↓
áp dụng các eligibility rule tương ứng
↓
tạo danh sách mã riêng
↓
tạo Target Portfolio riêng
↓
```

```
sau đó mới được đưa sang cùng backtest engine ở Bước 2
```

Do đó Bước 1 không tạo một Target Portfolio chung cho toàn bộ M3.

Nó tạo riêng:

```
Target Combined
Target Market-only
Target Financial-only
Target Equal-weight
Target Momentum-only
```

Điểm quan trọng:

không tồn tại một Financial eligibility gate chung
áp trước cả 5 strategy

Financial eligibility chỉ là prerequisite của strategy thực sự sử dụng Financial state theo contract tương ứng.

Cụ thể:

```
Combined
→ Market Eligible Pool ∩ Financial Eligible Pool

Market-only
→ Market Eligible Pool

Financial-only
→ Financial Eligible Pool ∩ traded universe

Equal-weight
→ traded universe

Momentum-only
→ traded universe ∩ mom_126 hợp lệ
```

Do đó, việc một ticker thiếu một hoặc nhiều thông tin tài chính sau:

```
Financial B label
→ nhãn cụm tài chính B

Financial PIT state
→ trạng thái dữ liệu tài chính hợp lệ theo nguyên tắc PIT tại decision_date

Financial indicator
→ các chỉ số tài chính cần thiết cho strategy

Financial sector coverage
→ ticker/ngành có được mô hình tài chính B hỗ trợ hay không
```

không đồng nghĩa với việc ticker đó bị loại khỏi tất cả strategy.

Cụ thể, việc thiếu các thông tin tài chính trên **không tự động loại ticker khỏi**:

```
Market-only
Equal-weight
Momentum-only
```

vì ba strategy này không yêu cầu Financial state làm điều kiện đầu vào.

Ví dụ, các doanh nghiệp thuộc:

```
Ngân hàng
Chứng khoán
Bảo hiểm
```

có thể không nằm trong phạm vi của mô hình Financial B, nhưng vẫn có thể tham gia:

```
Market-only
Equal-weight
Momentum-only
```

nếu ticker thỏa điều kiện universe riêng của strategy tương ứng.

Ngược lại:

```
Combined
Financial-only
```

có sử dụng Financial state, nên ticker phải đáp ứng các yêu cầu tài chính được quy định cho từng strategy.

Tổng quan cách chọn của 5 strategy (5 danh mục)

STT	Strategy (Danh mục)	Universe	Financial eligibility	Signal / Ranking	Top 30%	Trọng số
1	Combined	Market Eligible ∩ Financial Eligible	Có	Market criteria + Financial criteria	Có	Equal weight
2	Market-only	Market Eligible Pool	Không	Market criteria	Có	Equal weight
3	Financial-only	Financial Eligible Pool ∩ traded universe	Có	Financial criteria	Có	Equal weight
4	Equal-weight	traded universe	Không	Không ranking	Không	Equal weight toàn universe

5	Momentum-only	traded universe ∩ mom_126 hợp lệ	Không	mom_126	Có	Equal weight top 30%

9.1. STRATEGY 1 — Combined

`Combined` là strategy chính kết hợp Market và Financial.

Rule được sử dụng

```
Universe / eligibility
→ Mục 3.4.2 — Combined

Market criteria
→ Mục 3.5.1

Financial criteria
→ Mục 3.5.2

Missing Financial criteria
→ Mục 3.5.3

S_market / S_fin
→ Mục 3.5.4

Combined ranking / tie-break
→ Mục 3.5.5

Top-selection + equal weight
→ Mục 3.6

UNIVERSE_TOO_SMALL
→ Mục 3.7
```

Luồng riêng của Combined:

```
Market Eligible Pool
∩
Financial Eligible Pool
↓
Combined Candidate Universe
↓
Combined ranking
↓
Mục 3.6
↓
Target Portfolio
```

Combined là strategy sử dụng đồng thời cả Market và Financial branch.

Mọi rule còn ở trạng thái `PROPOSED/OPEN` tại Mục 3 vẫn giữ nguyên trạng thái đó; Mục 9 không tự nâng chúng thành methodology đã khóa.

9.2. STRATEGY 2 — Market-only

`Market-only` chỉ sử dụng Market state và Market signal để lựa chọn/xếp hạng cổ phiếu.

Rule được sử dụng

```
Universe / eligibility
→ Mục 3.4.3 — Market-only

Market criteria
→ Mục 3.5.1

Market ranking
→ Mục 3.5.4
→ score = S_market

Top-selection + equal weight
→ Mục 3.6

UNIVERSE_TOO_SMALL
→ Mục 3.7
```

Luồng riêng:

```
Market Eligible Pool
↓
Market ranking
↓
Mục 3.6
↓
Target Portfolio
```

Market-only không sử dụng Financial eligibility, Financial PIT, Financial criteria hoặc Financial ranking để tạo Target Portfolio.

Financial B coverage không được dùng để thu hẹp Market-only universe.

Mọi rule còn ở trạng thái `PROPOSED/OPEN` tại Mục 3 vẫn giữ nguyên trạng thái đó.

9.3. STRATEGY 3 — Financial-only

`Financial-only` chỉ sử dụng Financial signal để lựa chọn/xếp hạng cổ phiếu.

Rule được sử dụng

```
Universe / eligibility
→ Mục 3.4.4 — Financial-only

Financial criteria
→ Mục 3.5.2

Missing Financial criteria
→ Mục 3.5.3

Financial ranking
→ Mục 3.5.4
→ score = S_fin

Top-selection + equal weight
→ Mục 3.6

UNIVERSE_TOO_SMALL
→ Mục 3.7
```

Luồng riêng:

```
Financial Eligible Pool
∩
traded universe
↓
Financial-only Candidate Universe
↓
Financial ranking
↓
Mục 3.6
↓
Target Portfolio
```

Financial-only không yêu cầu mã nằm trong Market leading cluster và không sử dụng MC1–MC3 để tính ranking.

Market data ở strategy này chỉ tham gia ở mức cần thiết để xác định `traded universe`; việc lựa chọn/xếp hạng chính sử dụng Financial branch.

Mọi rule còn ở trạng thái `PROPOSED/OPEN` tại Mục 3 vẫn giữ nguyên trạng thái đó.

9.4. STRATEGY 4 — Equal-weight

`Equal-weight` là research control sử dụng toàn bộ `traded universe` hợp lệ.

Rule được sử dụng

```
Universe / eligibility
→ Mục 3.4.5 — Equal-weight
```

```
Equal-weight allocation
→ Mục 3.6.2
```

Luồng riêng:

```
traded universe
↓
lấy toàn bộ ticker hợp lệ
↓
Equal weight
↓
Target Portfolio
```

Equal-weight không sử dụng:

```
Market ranking
Financial ranking
Financial PIT
Financial eligibility
top 30%
```

và theo implementation hiện tại không áp dụng:

```
n_min
UNIVERSE_TOO_SMALL
```

Rank trong output, nếu có, chỉ phục vụ deterministic output ordering; không biểu thị chất lượng hoặc mức độ ưu tiên đầu tư.

9.5. STRATEGY 5 — Momentum-only

`Momentum-only` chỉ sử dụng `mom_126` để lựa chọn/xếp hạng cổ phiếu.

Rule được sử dụng

```
Universe + momentum ranking
→ Mục 3.4.5 — Momentum-only

Top-selection + equal weight
→ Mục 3.6
```

Luồng riêng:

```
traded universe
∩
mom_126 hợp lệ
```

```
↓
rank mom_126 giảm dần
↓
Mục 3.6
↓
Target Portfolio
```

Momentum-only không sử dụng:

```
Financial eligibility
Financial PIT
Financial criteria
Combined ranking
MC2 = downside_vol_63
MC3 = liquidity_21
```

để tính ranking.

Theo implementation hiện tại, Momentum-only không áp dụng:

```
n_min
UNIVERSE_TOO_SMALL
```

trừ khi strategy contract/config sau này được thay đổi bằng decision mới.

9.6. Điểm mà 5 strategy bắt đầu dùng chung logic

Sau khi Bước 1 tạo được:

```
Target Portfolio
```

thì cách lựa chọn cổ phiếu kết thúc.

Từ thời điểm này:

```
Combined
Market-only
Financial-only
Equal-weight
Momentum-only
```

đều được đưa vào:

```
cùng engine
```

ở Bước 2.

---

10. BƯỚC 2 — GIẢ LẬP MUA/BÁN THEO ENGINE GITHUB

10.1. Map `decision_date` sang `execution_date`

Rule đã khóa:

```
execution_date = phiên giao dịch hợp lệ đầu tiên sau decision_date
```

Runner dùng trading calendar và `bisect_right`, do đó bắt buộc:

```
execution_date > decision_date
```

10.2. Revalue Current Portfolio trước rebalance

Portfolio state gồm:

```
cash
values[ticker]
marks[ticker]
```

Trong đó:

```
cash
= số tiền mặt hiện đang có trong danh mục

values[ticker]
= giá trị hiện tại của phần vốn đang nắm giữ ở từng mã cổ phiếu

marks[ticker]
= mức giá gần nhất đã được dùng để định giá mã cổ phiếu đó
```

Tại `execution_date`, cập nhật lại giá trị của từng mã đang nắm giữ trước khi thực hiện giao dịch:

```
nếu current_price hợp lệ:
    values[ticker] *= current_price / marks[ticker]
    marks[ticker] = current_price

nếu current_price is None:
    values[ticker] giữ nguyên
    marks[ticker] giữ nguyên
    ghi nhận PRICE_STALE
```

Như vậy missing price không tự biến return thành 0; engine giữ nguyên giá trị đang được mark bằng giá hợp lệ gần nhất cho tới khi có giá mới hợp lệ.

Tính `pre_trade_NAV` sau khi revalue:

```
pre_trade_NAV
=
cash
+
Σ values[ticker]
```

NAV này là giá trị danh mục ngay trước rebalance.

`pre_weight` của từng ticker được tính:

```
pre_weight_i
=
values[ticker_i] / pre_trade_NAV
```

Đối với Cash:

```
cash_weight
=
cash / pre_trade_NAV
```

10.3. Current-vs-Target

Với union của mã đang nắm và mã trong target:

```
delta_weight_i = target_weight_i - pre_weight_i
```

- `delta < -EPS` → SELL;
- `delta > EPS` → BUY;
- `|delta| <= EPS` và mã còn trong target → HOLD.

Với EPS = 1e-12, là một ngưỡng kỹ thuật cực nhỏ
để tránh giao dịch do sai số tính toán.

10.4. Tính delta_weight và xác định SELL / BUY / HOLD

Với mỗi ticker:

```
delta_weight_i
=
target_weight_i - pre_weight_i
```

SELL nếu:

```
delta_weight_i < -EPS
```

thì:

```
side = SELL
```

Giá trị SELL dự kiến:

```
intended_sell_value_i
=
-delta_weight_i × pre_trade_NAV
```

Ví dụ:

```
pre_weight_FPT = 40%

target_weight_FPT = 30%

delta_weight_FPT = -10%

pre_trade_NAV = 100 triệu đồng
```

thì:

```
intended_sell_value_FPT
= 10 triệu đồng
```

BUY nếu:

```
delta_weight_i > EPS
```

thì:

```
side = BUY
```

Giá trị BUY dự kiến:

```
intended_buy_value_i
=
delta_weight_i × pre_trade_NAV
```

HOLD nếu:

```
|delta_weight_i| <= EPS
và ticker vẫn nằm trong Target:
```

thì:

```
side = HOLD
intended = 0
executed = 0
```

```
status = HOLD
```

Cần phân biệt:

```
pre_weight
target_weight
delta_weight
→ đơn vị tỷ trọng
```

và:

```
intended
executed
→ giá trị trong value space của Portfolio
```

10.5. Kiểm tra executability và thực hiện rebalance

10.5.1. Điều kiện executable — Q12

Một BUY hoặc SELL chỉ executable khi:

```
price(ticker, execution_date) is not None
AND
volume > 0
AND
ceiling/floor data tồn tại
AND
ceiling/floor rule PASS
```

Thiếu ceiling/floor data → không executable, flag `LIMIT_DATA_MISSING` (Mục 3.8.6).

Canonical tradability:

```
volume = 0
→ BUY_FAILED và SELL_FAILED

raw_close = ceiling
→ BUY_FAILED

raw_close = floor
→ SELL_FAILED
```

Không có đủ order-book/dư mua-dư bán nên không giả định lệnh vẫn khớp tại giá trần/sàn.

Ceiling/floor được xác định từ C8 raw market information của đúng ticker/phiên. Execution/valuation price basis của portfolio vẫn theo Q11 adj_close.

10.5.2. Thực hiện SELL trước

Engine xử lý toàn bộ SELL trước BUY.

Với mỗi SELL:

```
intended_sell_value
=
-delta_weight × pre_trade_NAV
```

Nếu executable:

```
values[ticker] -= intended_sell_value

cash += intended_sell_value

executed = intended_sell_value

status = EXECUTED
```

Nếu sau SELL:

```
values[ticker] <= EPS × pre_trade_NAV
```

thì:

```
xóa ticker khỏi values
xóa marks[ticker]
```

Nếu SELL không executable:

```
values[ticker] giữ nguyên
cash không tăng
executed = 0
status = SELL_FAILED
```

SELL_FAILED không giải phóng vốn.

10.5.3. Xác định Cash thực tế sau SELL

Sau khi xử lý hết SELL:

```
available_cash
=
Cash thực tế trong Portfolio
```

Nếu có SELL_FAILED:

```
available_cash
<
Cash dự kiến nếu mọi SELL thành công
```

Engine không dùng margin và không giả định vốn từ SELL_FAILED đã được giải phóng.

10.5.4. Kiểm tra BUY

Với mỗi BUY:

```
intended_buy_value
=
delta_weight × pre_trade_NAV
```

Nếu BUY không executable:

- executed = 0
- status = BUY_FAILED

BUY_FAILED:

- không làm giảm Cash
- không tạo position mới
- không chọn ticker khác thay thế

Chỉ BUY executable mới được tính vào:

```
required_buy_cash = Σ intended_buy_value của các BUY executable
```

10.5.5. Proportional scaling

Nếu:

```
required_buy_cash <= available_cash
```

thì:

```
execution_scale = 1
```

Nếu:

```
required_buy_cash > available_cash
```

thì:

```
execution_scale = available_cash / required_buy_cash
```

Tổng quát:

```
execution_scale = min(1, available_cash / required_buy_cash)
```

Tất cả BUY executable dùng cùng một execution_scale.

10.5.6. Thực hiện BUY

Với mỗi BUY executable:

```
executed_buy_value
=
intended_buy_value × execution_scale
```

Sau execution:

```
cash
-=
executed_buy_value
```

Nếu ticker chưa tồn tại:

```
values[ticker]
=
executed_buy_value

marks[ticker]
=
price(ticker, execution_date)
```

Nếu ticker đã tồn tại:

```
values[ticker]
+=
executed_buy_value
```

Nếu:

```
execution_scale = 1
```

thì:

```
status = EXECUTED
```

Nếu:

```
execution_scale < 1
```

thì:

```
status = SCALED
```

10.5.7. HOLD

HOLD không phát sinh giao dịch:

```
intended = 0
executed = 0
status = HOLD
```

Do đó:

- không thay đổi position
- không thay đổi Cash
- không tạo turnover

10.5.8. Missing price theo từng ngữ cảnh

Cần phân biệt:

VALUATION tại 12.2:

```
price is None
→ PRICE_STALE
→ giữ last valid valuation
```

SELL tại 12.5:

```
price is None / non-tradable
→ SELL_FAILED
```

BUY tại 12.5:

```
price is None / non-tradable
→ BUY_FAILED
```

10.6. SELL_FAILED và proportional scaling

SELL_FAILED không làm Cash giảm trực tiếp, nhưng ngăn vốn trong vị thế được chuyển thành Cash.

Với các BUY còn lại có thể thực hiện:

```
required_buy_cash = Σ intended_buy_value của BUY hợp lệ
execution_scale = min(1, available_cash / required_buy_cash)
```

Mỗi BUY hợp lệ:

```
executed_buy_value = intended_buy_value × execution_scale
```

Nếu `execution_scale < 1`, status của BUY là `SCALED`.

Một SELL_FAILED có thể tồn tại nhiều kỳ nếu mỗi kỳ Target vẫn muốn giảm/bỏ mã đó nhưng mỗi lệnh SELL mới tiếp tục thất bại. **Không carry lệnh SELL cũ**; mỗi kỳ sinh action mới từ Current-vs-Target.

10.7. Output canonical Bước 2

A. `trade_ledger_<strategy>.csv`

Schema canonical theo engine GitHub:

```
date
ticker
side
pre_weight
target_weight
intended
executed
scale
status
```

Trong đó `date` là `execution_date`.

Status tối thiểu:

```
EXECUTED
BUY_FAILED
SELL_FAILED
SCALED
HOLD
```

Không tạo `failed_executions.csv`; failed trade được lọc trực tiếp từ `status`.

B. `holdings_<strategy>.csv`

Schema theo implementation GitHub hiện hành:

```
date
ticker
weight
```

Ý nghĩa:

- mỗi row phản ánh tỷ trọng Actual Holdings ngay sau rebalance;
- `CASH` là một row;
- đối với cổ phiếu, `weight` được tính từ `values[ticker] / NAV`;
- đối với `CASH`, `weight = cash / NAV`;
- tổng weight phải xấp xỉ 1 theo tolerance;
- giá trị tuyệt đối của vị thế vẫn tồn tại trong internal Portfolio state thông qua `pf.values` và `pf.cash`, nhưng không được persist thành field `position_value` trong `holdings_<strategy>.csv` theo Git hiện

hành.

C. `turnover_<strategy>.csv`

Schema theo GitHub:

```
date
turnover
```

Công thức:

```
traded = executed_buy_value + executed_sell_value

turnover = traded / (2 × pre_trade_NAV)
```

Đây là one-way turnover theo owner decision hiện hành.

Failed/HOLD có:

```
executed = 0
```

nên không đóng góp turnover.

---

11. BƯỚC 3 — HOLDING PERIOD

Sau khi Bước 2 của kỳ t hoàn tất, Portfolio thực tế được giữ trong engine cho tới `execution_date` của kỳ kế tiếp.

Bước 3 không phải một hàm xử lý độc lập trong code.

Luồng:

```
Actual Holdings sau rebalance kỳ t
↓
giữ Portfolio state
↓
không rebalance giữa kỳ
↓
execution_date kỳ t+1
↓
revalue Current Portfolio theo Mục 10.2
↓
tiếp tục rebalance kỳ t+1
```

11.1. Portfolio state được carry sang kỳ tiếp theo

Sau rebalance, engine tiếp tục giữ:

```
cash
values[ticker]
marks[ticker]
```

Portfolio không được reset về Cash và không tạo Portfolio mới ở kỳ kế tiếp.

Không tạo file handoff riêng giữa hai kỳ; Portfolio state trong engine chính là trạng thái được carry forward.

11.2. Trong holding period

Từ `execution_date` kỳ t đến `execution_date` kỳ t+1, engine hiện tại:

- không revalue từng ngày;
- không tính NAV từng ngày;
- không tính weight từng ngày;
- không rebalance giữa kỳ;
- không kiểm tra missing valuation price từng ngày.

Do giá thị trường thay đổi, tỷ trọng kinh tế của các vị thế có thể drift so với tỷ trọng ngay sau rebalance.

Weight drift chỉ được phản ánh khi engine thực hiện `revalue()` tại kỳ kế tiếp.

11.3. Kết thúc holding period tại kỳ kế tiếp

Tại `execution_date` kỳ t+1, engine thực hiện lại Mục 10.2:

```
Current Portfolio
↓
revalue
↓
cập nhật values[ticker]
↓
cập nhật marks[ticker] khi có giá hợp lệ
↓
PRICE_STALE nếu thiếu giá
↓
pre_trade_NAV
```

Sau đó:

```
pre_weight
=
values[ticker] / pre_trade_NAV
```

và Current Portfolio mới được so sánh với Target Portfolio của kỳ t+1 để thực hiện rebalance.

Do đó:

```
NAV cuối holding period kỳ t
=
pre_trade_NAV trước rebalance kỳ t+1
```

Đây là cùng một NAV tại cùng một thời điểm, không phải hai phép tính riêng.

11.4. Kỳ cuối của segment

Nếu không còn kỳ rebalance tiếp theo, engine sử dụng `end_date` theo Quy tắc 3.9 để chốt valuation cuối segment.

```
Current Portfolio
↓
revalue tại end_date
↓
PRICE_STALE nếu thiếu giá
↓
NAV cuối segment
```

Tại `end_date` :

```
không rebalance
không tính delta_weight mới
không BUY
không SELL
```

`end_date` chỉ dùng để chốt giá trị cuối danh mục.

11.5. Delisting — phần mở rộng

Main M3 không suy `missing price → delisted` . Runtime hiện dùng `PRICE_STALE` khi không có valuation price mới.

```
DELISTED_EXIT
= FUTURE EXTENSION
= CHƯA IMPLEMENT
```

Chỉ xem xét `DELISTED_EXIT` sau khi có PIT-safe historical security-status/delisting contract tối thiểu gồm:

- `event_type`
- `available_at`
- `announcement_date`
- `last_trading_date`
- `effective_date`
- `source` / provenance

Không được retroactively giả định đã biết phiên cuối chỉ vì ticker biến mất khỏi dữ liệu giá.

12. BƯỚC 4 — CHUYỂN SANG KỲ TIẾP THEO

Bước 4 biểu diễn việc engine chuyển sang phần tử tiếp theo của `schedule` ; không phải một hàm xử lý độc lập.

Portfolio state được carry sang kỳ tiếp theo theo Mục 11.

Mục này chỉ bổ sung hai nguyên tắc khi bắt đầu kỳ mới.

12.1. Target Portfolio phải được tính lại

Tại `decision_date` của kỳ t+1, strategy quay lại Bước 1 và tạo Target Portfolio mới từ thông tin hợp lệ tại thời điểm đó.

```
decision_date t+1
↓
Candidate Universe mới
↓
ranking / selection mới
↓
Target Portfolio mới
```

Không carry Target Portfolio của kỳ trước sang làm Target của kỳ mới.

Actual Holdings kỳ trước chỉ trở thành Current Portfolio để so sánh với Target mới tại Bước 2.

12.2. Không carry lệnh giao dịch cũ

Các action:

```
BUY
SELL
HOLD
BUY_FAILED
SELL_FAILED
SCALED
```

chỉ thuộc rebalance đã sinh ra chúng.

Sang kỳ mới:

```
Current Portfolio
+
Target Portfolio mới
↓
tính lại delta_weight
↓
sinh action mới
```

Không carry BUY/SELL cũ và không tự retry một failed order cũ.

Ví dụ:

kỳ t:

```
VCB → SELL_FAILED
→ VCB vẫn còn trong Current Portfolio
```

Nếu ở kỳ t+1 Target mới vẫn yêu cầu:

```
VCB = 0%
```

thì Current Portfolio và Target mới tạo ra một SELL mới. Đây không phải lệnh SELL cũ được carry sang.

Chu trình tiếp tục theo Mục 7 cho tới hết segment; xử lý kỳ cuối tuân theo Mục 11.4 và Quy tắc 3.9.

---

13. CHẠY 4 BƯỚC TRÊN DEVELOPMENT VÀ HOLDOUT HIỆN HÀNH

13.1. Development

```
2019-01-31 → 2023-12-29
60 decision snapshots
```

Chạy cùng frozen config/engine. Sharpe không được báo cáo.

13.2. Holdout

```
2024-01-31 → 2025-12-31
24 decision snapshots
```

Chỉ chạy bằng đúng rule đã khóa trước khi mở Holdout. Sharpe không được báo cáo.

Tuy nhiên, upstream clustering model A/B đã từng được đánh giá
trên dữ liệu bao gồm giai đoạn 2024–2025 trước khi M3-A chạy.

Vì vậy Holdout phải được mô tả là:

- tách biệt đối với portfolio-rule evaluation;
- không dùng để tune strategy dựa trên return;
- nhưng không được mô tả là hoàn toàn unseen đối với toàn bộ
  quá trình lựa chọn upstream clustering model.

Final Evaluation sau 31/12/2025 là lớp đánh giá tiếp theo

sau khi strategy đã được freeze.

Mỗi segment có run directory độc lập:

```
data/experiments/m3/<segment>/<run_id>/
```

60 và 24 là decision snapshot count, không đồng nghĩa mọi target đều có execution trong segment:

```
decision_count = số decision snapshots
execution_count = số decision có execution_date hợp lệ trong cùng segment
valuation_end_date = phiên giao dịch cuối cùng của segment
```

---

14. CÁC STRATEGY ĐƯỢC CHẠY TRONG CÙNG OUTPUT CONTRACT

14.1. Combined — chiến lược chính

Universe:

```
Market Eligible Pool
∩
Financial Eligible Pool
```

Sử dụng Market + Financial signal/ranking theo config đã khóa.

Nếu đủ điều kiện, chọn top 30% và phân bổ equal weight.

14.2. Market-only — đối chứng

Universe:

```
Market Eligible Pool
```

Chỉ sử dụng Market signal/ranking.

Không yêu cầu Financial eligibility hoặc Financial B coverage.

Nếu đủ điều kiện, chọn top 30% và phân bổ equal weight.

14.3. Financial-only — đối chứng

Universe:

```
Financial Eligible Pool
∩
```

```
traded universe
```

Chỉ sử dụng Financial signal/ranking.

Không sử dụng Market leading cluster làm điều kiện lựa chọn.

Nếu đủ điều kiện, chọn top 30% và phân bổ equal weight.

14.4. Equal-weight — đối chứng

Universe:

```
traded universe
```

Không sử dụng Market/Financial ranking.

Không chọn top 30%.

Toàn bộ universe được phân bổ equal weight.

14.5. Momentum-only — đối chứng

Universe:

```
traded universe
∩
mom_126 hợp lệ
```

Ranking:

```
mom_126
```

Chọn top 30% và phân bổ equal weight.

Không yêu cầu Financial eligibility hoặc Financial B coverage.

Tên strategy từ config được dùng trực tiếp trong output suffix.

VNINDEX là benchmark chính thức, không phải strategy.

---

15. VÍ DỤ MINH HỌA CHI TIẾT QUA 3 KỲ

Ví dụ này chỉ minh họa mechanics của backtest, không phải kết quả nghiên cứu thật.

Đơn vị của:

```
cash
values[ticker]
```

```
intended
executed
NAV
```

trong ví dụ là:

```
triệu đồng
```

Để dễ tính toán, giả sử danh mục ban đầu có:

```
cash = 100 triệu đồng
```

Trong code thật, engine khởi tạo:

```
cash = 1.0
```

Việc dùng `100 triệu đồng` chỉ giúp minh họa dễ đọc hơn; tỷ trọng và logic không thay đổi.

15.1. Lịch 3 kỳ

```
Kỳ 1:
decision_date  = 31/01/2024
execution_date = 01/02/2024

Kỳ 2:
decision_date  = 29/02/2024
execution_date = 01/03/2024

Kỳ 3:
decision_date  = 29/03/2024
execution_date = 01/04/2024

end_date = 29/04/2024
```

Trong cả ba kỳ, giả sử sau khi thực hiện Market–Financial PIT và eligibility:

```
N_candidates = 11
```

Do đó:

```
n_select
=
ceil(0.30 × 11)
=
4
```

Vì danh mục phân bổ equal-weight:

```
target_weight
=
1 / 4
=
25%
```

Mỗi kỳ Target Portfolio chính thức gồm đúng:

```
4 mã
×
25%
```

KỲ 1

BƯỚC 1 — CHỌN CỔ PHIẾU

1.1. Financial PIT

Ví dụ FPT có:

```
FY2022:
usable_from_date = 15/03/2023

FY2023:
usable_from_date = 20/02/2024
```

Tại:

```
decision_date = 31/01/2024
```

ta có:

```
15/03/2023 <= 31/01/2024

20/02/2024 > 31/01/2024
```

Vì vậy:

```
usable_financial_years
=
{2022}
```

và:

```
selected_fiscal_year
=
2022
```

FY2023 chưa được sử dụng vì tại `decision_date` nó chưa đạt `usable_from_date`.

Giả sử CTG không có bất kỳ fiscal year nào thỏa:

```
usable_from_date <= decision_date
```

thì:

```
CTG
→ FINANCIAL_NOT_AVAILABLE
```

và CTG không thể đi vào Financial Eligible Pool ở kỳ này.

1.2. Candidate Universe

Giả sử:

```
Market Eligible Pool
=
{
 FPT, HPG, MWG, VNM, VCB, SSI,
 GAS, VIC, STB, MSN, VRE,
 CTG, DGC
}
```

Financial Eligible Pool:

```
{
 FPT, HPG, MWG, VNM, VCB, SSI,
 GAS, VIC, STB, MSN, VRE,
 SAB
}
```

Candidate Universe:

```
Candidate Universe
=
Market Eligible Pool
∩
Financial Eligible Pool
```

suy ra:

```
{
 FPT,
 HPG,
 MWG,
 VNM,
 VCB,
 SSI,
 GAS,
 VIC,
 STB,
 MSN,
 VRE
}
```

Do đó:

```
N_candidates = 11
```

1.3. Combined Ranking

Ví dụ sau sử dụng công thức MC1–MC3, FC1–FC3
và Combined Ranking đã khóa tại Mục 3.

Giả sử sau khi chạy rule chính thức, kết quả ranking là:

rank	**ticker**
1	FPT
2	HPG
3	MWG
4	VCB
5	VNM
6	SSI
7	GAS
8	VIC
9	STB
10	MSN
11	VRE

Số mã chọn:

```
n_select
=
ceil(0.30 × 11)
=
4
```

Mỗi mã:

```
target_weight
=
1 / 4
=
0.25
=
25%
```

Target Portfolio kỳ 1:

decision_date	ticker	rank	target_weight	candidates	status
31/01/2024	FPT	1	0.25	11	OK
31/01/2024	HPG	2	0.25	11	OK
31/01/2024	MWG	3	0.25	11	OK
31/01/2024	VCB	4	0.25	11	OK

BƯỚC 2 — GIẢ LẬP MUA/BÁN KỲ 1

2.1. Portfolio ban đầu

Trước execution đầu tiên:

```
cash = 100

values = {}

marks = {}
```

Do chưa có cổ phiếu:

```
revalue()
→ không có ticker nào để cập nhật
```

Vì vậy:

```
pre_trade_NAV = 100
```

Pre-weight:

```
pre_weight_FPT = 0
pre_weight_HPG = 0
pre_weight_MWG = 0
pre_weight_VCB = 0
```

2.2. Giá tại 01/02/2024

Giả sử:

ticker	current_price	volume	Trạng thái	BUY executable?
FPT	100	> 0	bình thường	Có
HPG	50	> 0	bình thường	Có
MWG	40	> 0	đóng cửa tại giá trần	Không
VCB	80	> 0	bình thường	Có

MWG vẫn có price nhưng:

```
BUY MWG
→ không executable do giá trần
→ BUY_FAILED
```

2.3. Current-vs-Target

FPT

```
target_weight_FPT
=
25%

pre_weight_FPT
=
0%
```

Do đó:

```
delta_weight_FPT
=
25% - 0%
=
+25%
```

Vì:

```
delta_weight_FPT > 0
```

nên:

```
side = BUY
```

Giá trị BUY dự kiến:

```
intended_buy_value_FPT
=
delta_weight_FPT × pre_trade_NAV


=
25% × 100
=
25 triệu
```

Tương tự:

```
intended_buy_value_HPG = 25
intended_buy_value_MWG = 25
intended_buy_value_VCB = 25
```

2.4. Kiểm tra BUY

MWG không executable.

Do đó chỉ các BUY executable được tính vào:

```
required_buy_cash
=
25 FPT
+
25 HPG
+
25 VCB
=
75
```

Trong khi:

```
available_cash = 100
```

Do đó:

```
execution_scale
=
min(
    1,
    available_cash / required_buy_cash
)


=
min(
    1,
    100 / 75
)
=
1
```

Không cần scaling.

2.5. Execution kỳ 1

ticker	side	pre_weight	target_weight	intended	executed	scale	status
FPT	BUY	0	0.25	25	25	1	EXECUTED
HPG	BUY	0	0.25	25	25	1	EXECUTED
MWG	BUY	0	0.25	25	0	1	BUY_FAILED
VCB	BUY	0	0.25	25	25	1	EXECUTED

Portfolio sau giao dịch:

```
values[FPT] = 25
marks[FPT]  = 100

values[HPG] = 25
marks[HPG]  = 50

values[VCB] = 25
marks[VCB]  = 80

cash = 25
```

MWG:

```
không tạo values[MWG]
không tạo marks[MWG]
```

vì BUY thất bại.

2.6. Actual Holdings kỳ 1

Theo schema GitHub hiện hành, `holdings_<strategy>.csv` chỉ persist:

```
date
ticker
weight
```

Output holdings tại `execution_date = 01/02/2024` :

date	ticker	weight
01/02/2024	FPT	25%
01/02/2024	HPG	25%
01/02/2024	VCB	25%
01/02/2024	CASH	25%

Giá trị tuyệt đối của các vị thế vẫn được engine giữ trong internal Portfolio state qua `values[ticker]` và `cash` ; chúng không được persist thành field riêng trong holdings output.

Mặc dù Target muốn MWG 25%, Actual Portfolio giữ:

```
25% Cash
```

do:

```
MWG BUY_FAILED
```

2.7. Turnover kỳ 1

Executed BUY:

```
FPT = 25
HPG = 25
VCB = 25
```

Tổng:

```
traded
=
75
```

Turnover:

```
turnover
=
75 / (2 × 100)
=
0.375
=
37.5%
```

BƯỚC 3 — HOLDING PERIOD KỲ 1

Sau rebalance:

```
cash = 25

values:
FPT = 25
HPG = 25
VCB = 25

marks:
FPT = 100
HPG = 50
VCB = 80
```

Trong holding period:

```
không revalue mỗi ngày
không tính NAV mỗi ngày
không tính weight mỗi ngày
không rebalance giữa kỳ
không kiểm tra missing price mỗi ngày
```

Portfolio state chỉ được giữ trong RAM.

BƯỚC 4 — CHUYỂN SANG KỲ 2

Không tạo Portfolio mới.

Không reset:

```
cash
values
marks
```

MWG từng:

```
BUY_FAILED
```

nhưng lệnh BUY cũ không được carry sang kỳ 2.

Kỳ 2 quay lại Bước 1 và tạo Target mới.

KỲ 2

BƯỚC 1 — TARGET MỚI

Tại:

```
decision_date = 29/02/2024
```

FY2023 của FPT đã đạt:

```
usable_from_date = 20/02/2024
```

vì:

```
20/02/2024 <= 29/02/2024
```

nên:

```
usable_financial_years
=
{2022, 2023}
```

và:

```
selected_fiscal_year
=
2023
```

Như vậy Financial state của cùng một mã có thể thay đổi giữa hai kỳ theo PIT.

Ranking kỳ 2

Giả sử Candidate Universe vẫn có:

```
11 mã
```

Ranking mới:

rank	ticker
1	FPT
2	MWG
3	VNM
4	SSI
5	HPG
6	VCB
7	GAS
8	VIC
9	STB
10	MSN
11	VRE

Target kỳ 2:

```
FPT = 25%
MWG = 25%
VNM = 25%
SSI = 25%
```

HPG và VCB không còn trong Target:

```
target_weight_HPG = 0
target_weight_VCB = 0
```

BƯỚC 2 — REVALUE KỲ 2

Ngày:

```
execution_date = 01/03/2024
```

Giả sử:

```
FPT current_price = 120
HPG current_price = None
VCB current_price = 88
```

FPT

Trước đó:

```
values[FPT] = 25
marks[FPT] = 100
```

Revalue:

```
values[FPT]
=
25 × 120 / 100
=
30
```

Sau đó:

```
marks[FPT] = 120
```

HPG — missing price

```
current_price = None
```

Do đó:

```
values[HPG] giữ nguyên = 25

marks[HPG] giữ nguyên = 50

ghi nhận:
PRICE_STALE(01/03/2024, HPG)
```

Không đặt:

```
values[HPG] = 0
```

VCB

```
values[VCB]
=
25 × 88 / 80
=
27.5
```

Sau đó:

```
marks[VCB] = 88
```

Cash

```
cash = 25
```

---

pre_trade_NAV kỳ 2

```
pre_trade_NAV
=
30
+
25
+
27.5
+
25
=
107.5
```

Do đó:

```
NAV = 107.5 triệu đồng
```

PRE_WEIGHT KỲ 2

FPT:

```
pre_weight_FPT
```

```
=
30 / 107.5
≈
27.9070%
```

HPG:

```
pre_weight_HPG
=
25 / 107.5
≈
23.2558%
```

VCB:

```
pre_weight_VCB
=
27.5 / 107.5
≈
25.5814%
```

Cash:

```
25 / 107.5
≈
23.2558%
```

Đây là weight drift so với Actual Holdings ngay sau kỳ 1.

CURRENT VS TARGET KỲ 2

Engine không cần tạo một biến trung gian chứa giá trị tiền mục tiêu tuyệt đối.

Mỗi action được xác định trực tiếp từ:

```
delta_weight_i
=
target_weight_i - pre_weight_i
```

Sau đó:

```
BUY:
intended_buy_value_i
=
delta_weight_i × pre_trade_NAV
```

hoặc:

```
SELL:
intended_sell_value_i
=
-delta_weight_i × pre_trade_NAV
```

FPT — SELL bớt

```
target_weight_FPT
=
25%

pre_weight_FPT
≈
27.9070%
```

Do đó:

```
delta_weight_FPT
=
25% - 27.9070%
≈
-2.9070%
```

Vì:

```
delta_weight_FPT < 0
```

nên:

```
side = SELL
```

Giá trị SELL dự kiến:

```
intended_sell_value_FPT
=
-delta_weight_FPT × pre_trade_NAV


≈
2.9070% × 107.5
=
3.125 triệu
```

HPG — SELL toàn bộ

```
target_weight_HPG = 0%

pre_weight_HPG
≈
23.2558%
```

Do đó:

```
delta_weight_HPG
=
0% - 23.2558%
=
-23.2558%


intended_sell_value_HPG
=
-delta_weight_HPG × pre_trade_NAV


≈
23.2558% × 107.5
=
25 triệu
```

VCB — SELL toàn bộ

```
target_weight_VCB
=
0%

pre_weight_VCB
≈
25.5814%


delta_weight_VCB
=
0% - 25.5814%
=
-25.5814%
```

Do đó:

```
intended_sell_value_VCB
=
-delta_weight_VCB × pre_trade_NAV
```

```
≈
25.5814% × 107.5
=
27.5 triệu
```

MWG — BUY

```
pre_weight_MWG = 0%

target_weight_MWG = 25%


delta_weight_MWG
=
25% - 0%
=
+25%
```

Do đó:

```
intended_buy_value_MWG
=
delta_weight_MWG × pre_trade_NAV


=
25% × 107.5
=
26.875 triệu
```

VNM — BUY

```
pre_weight_VNM = 0%

target_weight_VNM = 25%


delta_weight_VNM = +25%
```

Do đó:

```
intended_buy_value_VNM
=
25% × 107.5
=
26.875 triệu
```

SSI — BUY

```
pre_weight_SSI = 0%

target_weight_SSI = 25%


delta_weight_SSI = +25%
```

Do đó:

```
intended_buy_value_SSI
=
25% × 107.5
=
26.875 triệu
```

SELL ĐƯỢC THỰC HIỆN TRƯỚC

FPT

Giả sử SELL executable:

```
executed_sell_value_FPT
=
3.125


status = EXECUTED
```

Sau SELL:

```
values[FPT]
=
30 - 3.125
=
26.875
```

Cash:

```
cash
=
25 + 3.125
```

```
=
28.125
```

HPG

HPG không có valid price:

```
price(HPG, 01/03/2024)
=
None
```

Do đó:

```
SELL_FAILED


executed = 0
```

HPG vẫn:

```
values[HPG] = 25
marks[HPG] = 50
```

Cash không nhận được 25 triệu dự kiến.

VCB

Giả sử SELL executable:

```
executed_sell_value_VCB
=
27.5


status = EXECUTED
```

VCB bị bán hết:

```
xóa values[VCB]

xóa marks[VCB]
```

Cash trở thành:

```
cash
```

```
=
28.125 + 27.5
=
55.625
```

BUY VÀ PROPORTIONAL SCALING KỲ 2

Ba BUY đều executable:

```
intended_buy_value_MWG = 26.875

intended_buy_value_VNM = 26.875

intended_buy_value_SSI = 26.875
```

Tổng:

```
required_buy_cash
=
26.875
+
26.875
+
26.875
=
80.625
```

Nhưng:

```
available_cash
=
55.625
```

Do:

```
HPG SELL_FAILED
```

không giải phóng 25 triệu Cash.

Do đó:

```
execution_scale
=
available_cash / required_buy_cash


=
```

```
55.625 / 80.625
≈
0.689922
```

Mỗi BUY executable:

```
executed_buy_value
=
intended_buy_value × execution_scale
```

MWG:

```
executed_buy_value_MWG
=
26.875 × 0.689922
≈
18.541667
```

VNM:

```
executed_buy_value_VNM
≈
18.541667
```

SSI:

```
executed_buy_value_SSI
≈
18.541667
```

LEDGER KỲ 2

ticker	side	pre_weight	target_weight	intended	executed	scale	status
FPT	SELL	27.9070%	25%	3.125	3.125	1	EXECUTED
HPG	SELL	23.2558%	0%	25	0	1	SELL_FAILED
VCB	SELL	25.5814%	0%	27.5	27.5	1	EXECUTED
MWG	BUY	0%	25%	26.875	18.541667	0.689922	SCALED
VNM	BUY	0%	25%	26.875	18.541667	0.689922	SCALED
SSI	BUY	0%	25%	26.875	18.541667	0.689922	SCALED

ACTUAL HOLDINGS KỲ 2

Theo schema GitHub hiện hành, output holdings tại `execution_date = 01/03/2024` là:

date	ticker	weight
01/03/2024	FPT	25.0000%
01/03/2024	HPG	23.2558%
01/03/2024	MWG	17.2481%
01/03/2024	VNM	17.2481%
01/03/2024	SSI	17.2481%
01/03/2024	CASH	0%

Internal Portfolio state tương ứng vẫn giữ:

```
values[FPT] = 26.875000
values[HPG] = 25.000000
values[MWG] = 18.541667
values[VNM] = 18.541667
values[SSI] = 18.541667
cash = 0
```

NAV tại thời điểm này:

```
107.5 triệu
```

Actual Portfolio khác Target vì:

```
HPG SELL_FAILED
+
BUY bị proportional scaling
```

TURNOVER KỲ 2

Executed SELL:

```
FPT = 3.125
VCB = 27.5
```

Tổng:

```
executed_sell_value
=
30.625
```

Executed BUY:

```
MWG = 18.541667
VNM = 18.541667
SSI = 18.541667
```

Tổng:

```
executed_buy_value
=
55.625
```

Do đó:

```
traded
=
30.625
+
55.625
=
86.25
```

Turnover:

```
turnover
=
86.25 / (2 × 107.5)
≈
0.401163
=
40.1163%
```

HPG không đóng góp turnover vì:

```
SELL_FAILED
executed = 0
```

BƯỚC 3 — HOLDING PERIOD KỲ 2

State được carry:

```
cash = 0
```

```
values:
FPT = 26.875
HPG = 25
MWG = 18.541667
VNM = 18.541667
SSI = 18.541667

marks:
FPT = 120
HPG = 50
MWG = 40
VNM = 80
SSI = 30
```

HPG vẫn:

```
marks[HPG] = 50
```

vì tại kỳ 2 không có giá mới hợp lệ.

BƯỚC 4 — CHUYỂN SANG KỲ 3

Kỳ 2:

```
SELL HPG
→ SELL_FAILED
```

Nhưng engine không carry lệnh SELL cũ sang kỳ 3.

Kỳ 3 quay lại Bước 1 và tạo Target mới.

KỲ 3

BƯỚC 1 — TARGET MỚI

Giả sử Candidate Universe tiếp tục có:

```
11 mã
```

Ranking:

rank	ticker
1	FPT

2	HPG
3	MWG
4	VCB
5	VNM
6	SSI
7	GAS
8	VIC
9	STB
10	MSN
11	VRE

Target kỳ 3:

```
FPT = 25%
HPG = 25%
MWG = 25%
VCB = 25%
```

Kỳ trước:

```
HPG target = 0
→ SELL_FAILED
```

Nhưng kỳ này:

```
HPG target = 25%
```

Do đó engine không tự retry SELL cũ.

Action mới phải được tính lại từ:

```
Current Portfolio
vs
Target Portfolio mới
```

BƯỚC 2 — REVALUE KỲ 3

Tại:

```
execution_date = 01/04/2024
```

giả sử:

```
FPT:
marks = 120
current_price = 120

HPG:
marks = 50
current_price = 55

MWG:
marks = 40
current_price = 40

VNM:
marks = 80
current_price = 80

SSI:
marks = 30
current_price = 30
```

FPT

```
values[FPT]
=
26.875 × 120 / 120
=
26.875


marks[FPT] = 120
```

HPG — giá xuất hiện trở lại sau PRICE_STALE

Trước đó:

```
marks[HPG] = 50
```

Tại `execution_date` kỳ 3:

```
current_price_HPG = 55
```

Do đó:

```
values[HPG]
=
25 × 55 / 50
=
27.5
```

Sau đó:

```
marks[HPG] = 55
```

Biến động từ last valid mark `50` đến giá mới `55` được phản ánh tại lần revalue này.

MWG

```
values[MWG]
=
18.541667 × 40 / 40
=
18.541667


marks[MWG] = 40
```

VNM

```
values[VNM]
=
18.541667 × 80 / 80
=
18.541667
```

SSI

```
values[SSI]
=
18.541667 × 30 / 30
=
18.541667
```

---

pre_trade_NAV KỲ 3

```
pre_trade_NAV
=
26.875
+
27.5
+
18.541667
+
18.541667
+
18.541667
=
110
```

Do đó:

```
NAV = 110 triệu
```

PRE_WEIGHT KỲ 3

FPT:

```
pre_weight_FPT
=
26.875 / 110
≈
24.4318%
```

HPG:

```
pre_weight_HPG
=
27.5 / 110
=
25%
```

MWG:

```
pre_weight_MWG
=
18.541667 / 110
≈
16.8561%
```

VNM:

```
pre_weight_VNM
≈
16.8561%
```

SSI:

```
pre_weight_SSI
≈
16.8561%
```

VCB:

```
pre_weight_VCB
=
0%
```

CURRENT VS TARGET KỲ 3

Target:

```
FPT = 25%
HPG = 25%
MWG = 25%
VCB = 25%

VNM = 0%
SSI = 0%
```

Engine tiếp tục dùng:

```
delta_weight_i
=
target_weight_i - pre_weight_i
```

HPG — HOLD

```
target_weight_HPG
=
25%

pre_weight_HPG
=
```

```
25%
```

Do đó:

```
delta_weight_HPG
=
25% - 25%
=
0
```

Vì:

```
|delta_weight_HPG| <= EPS
```

và HPG vẫn nằm trong Target:

```
side = HOLD

intended = 0

executed = 0

status = HOLD
```

Điều này cũng minh họa:

```
Kỳ 2:
HPG → SELL_FAILED

Kỳ 3:
HPG → HOLD
```

Lệnh SELL cũ không được carry sang.

FPT — BUY thêm

```
target_weight_FPT
=
25%

pre_weight_FPT
≈
24.4318%
```

Do đó:

```
delta_weight_FPT
```

```
=
25% - 24.4318%
≈
+0.5682%
```

Vì:

```
delta_weight_FPT > 0
```

nên:

```
side = BUY
```

Giá trị BUY dự kiến:

```
intended_buy_value_FPT
=
delta_weight_FPT × pre_trade_NAV


≈
0.5682% × 110
=
0.625 triệu
```

MWG — BUY thêm

```
target_weight_MWG
=
25%

pre_weight_MWG
≈
16.8561%
```

Do đó:

```
delta_weight_MWG
=
25% - 16.8561%
≈
+8.1439%
```

Giá trị BUY dự kiến:

```
intended_buy_value_MWG
=
```

```
delta_weight_MWG × pre_trade_NAV


≈
8.1439% × 110
=
8.958333 triệu
```

VNM — SELL toàn bộ

```
target_weight_VNM
=
0%

pre_weight_VNM
≈
16.8561%
```

Do đó:

```
delta_weight_VNM
=
0% - 16.8561%
=
-16.8561%
```

Giá trị SELL dự kiến:

```
intended_sell_value_VNM
=
-delta_weight_VNM × pre_trade_NAV


≈
16.8561% × 110
=
18.541667 triệu
```

Giả sử SELL executable:

```
executed
=
18.541667

status
=
EXECUTED
```

VNM bị bán hết:

```
xóa values[VNM]

xóa marks[VNM]
```

SSI — SELL nhưng không executable

```
target_weight_SSI
=
0%

pre_weight_SSI
≈
16.8561%


delta_weight_SSI
=
0% - 16.8561%
=
-16.8561%
```

Do đó:

```
intended_sell_value_SSI
=
-delta_weight_SSI × pre_trade_NAV


≈
16.8561% × 110
=
18.541667 triệu
```

Giả sử tại `execution_date` :

```
price hợp lệ

volume > 0

raw_close = floor
```

SELL không executable:

```
executed = 0

status = SELL_FAILED
```

SSI vẫn:

```
values[SSI]
=
18.541667
```

VCB — BUY mới

VCB hiện không nằm trong danh mục:

```
pre_weight_VCB
=
0%
```

Target:

```
target_weight_VCB
=
25%
```

Do đó:

```
delta_weight_VCB
=
25% - 0%
=
+25%
```

Giá trị BUY dự kiến:

```
intended_buy_value_VCB
=
delta_weight_VCB × pre_trade_NAV


=
25% × 110
=
27.5 triệu
```

Giả sử VCB executable.

SELL ĐƯỢC THỰC HIỆN TRƯỚC

VNM SELL thành công:

```
cash
=
18.541667
```

SSI:

```
SELL_FAILED
```

nên không tạo thêm Cash.

Do đó:

```
available_cash
=
18.541667
```

PROPORTIONAL SCALING KỲ 3

Các BUY executable:

```
intended_buy_value_FPT
=
0.625

intended_buy_value_MWG
=
8.958333

intended_buy_value_VCB
=
27.5
```

Do đó:

```
required_buy_cash
=
0.625
+
8.958333
+
27.5
=
37.083333
```

Trong khi:

```
available_cash
=
18.541667
```

Nên:

```
execution_scale
=
available_cash / required_buy_cash


=
18.541667 / 37.083333
=
0.5
```

Tất cả BUY executable dùng chung:

```
execution_scale = 0.5
```

FPT

```
executed_buy_value_FPT
=
intended_buy_value_FPT × execution_scale


=
0.625 × 0.5
=
0.3125
```

Sau BUY:

```
values[FPT]
=
26.875 + 0.3125
=
27.1875
```

MWG

```
executed_buy_value_MWG
=
```

```
8.958333 × 0.5
=
4.479167
```

Sau BUY:

```
values[MWG]
=
18.541667 + 4.479167
=
23.020834
```

VCB

```
executed_buy_value_VCB
=
27.5 × 0.5
=
13.75
```

VCB chưa tồn tại trong Portfolio trước BUY này.

Giả sử tại:

```
execution_date = 01/04/2024
```

giá VCB mà `price()` trả về là:

```
current_price_VCB
=
price(VCB, 01/04/2024)
=
100
```

Do đó sau BUY:

```
values[VCB]
=
13.75
```

và:

```
marks[VCB]
=
current_price_VCB
=
100
```

`marks[VCB] = 100` trở thành giá gần nhất dùng để định giá VCB cho lần `revalue()` tiếp theo.

LEDGER KỲ 3

ticker	side	pre_weight	target_weight	intended	executed	scale	status
VNM	SELL	16.8561%	0%	18.541667	18.541667	1	EXECUTED
SSI	SELL	16.8561%	0%	18.541667	0	1	SELL_FAILED
HPG	HOLD	25%	25%	0	0	1	HOLD
FPT	BUY	24.4318%	25%	0.625	0.3125	0.5	SCALED
MWG	BUY	16.8561%	25%	8.958333	4.479167	0.5	SCALED
VCB	BUY	0%	25%	27.5	13.75	0.5	SCALED

ACTUAL HOLDINGS SAU KỲ 3

Theo schema GitHub hiện hành, output holdings tại `execution_date = 01/04/2024` là:

date	ticker	weight
01/04/2024	FPT	24.7159%
01/04/2024	HPG	25.0000%
01/04/2024	MWG	20.9280%
01/04/2024	SSI	16.8561%
01/04/2024	VCB	12.5000%
01/04/2024	CASH	0%

Internal Portfolio state tương ứng vẫn giữ:

```
values[FPT] = 27.187500
values[HPG] = 27.500000
values[MWG] = 23.020833
values[SSI] = 18.541667
values[VCB] = 13.750000
cash = 0
```

NAV tại thời điểm này:

```
110 triệu
```

Tổng weight:

```
100%
```

Actual Holdings vẫn khác Target do:

```
SSI SELL_FAILED
+
các BUY bị proportional scaling
```

TURNOVER KỲ 3

Executed SELL:

```
VNM
=
18.541667
```

SSI:

```
SELL_FAILED

executed = 0
```

Executed BUY:

```
FPT = 0.3125

MWG = 4.479167

VCB = 13.75
```

Tổng BUY:

```
18.541667
```

Tổng:

```
traded
=
18.541667
```

```
+
18.541667
=
37.083334
```

Turnover:

```
turnover
=
37.083334 / (2 × 110)
≈
0.168561
=
16.8561%
```

BƯỚC 3 — HOLDING PERIOD SAU KỲ 3

Portfolio state:

```
cash = 0

values:
FPT = 27.1875
HPG = 27.5
MWG = 23.020833
SSI = 18.541667
VCB = 13.75

marks:
FPT = 120
HPG = 55
MWG = 40
SSI = 30
VCB = 100
```

Không còn kỳ rebalance thứ tư trong ví dụ.

Danh mục được giữ tới:

```
end_date = 29/04/2024
```

END_DATE — CHỐT NAV CUỐI

Giả sử tại:

```
end_date = 29/04/2024
```

có:

ticker	marks hiện tại	current_price
FPT	120	132
HPG	55	None
MWG	40	44
SSI	30	27
VCB	100	105

FPT

```
values[FPT]
=
27.1875 × 132 / 120
=
29.90625
```

Sau đó:

```
marks[FPT] = 132
```

HPG — missing price tại end_date

```
current_price
=
None
```

Do đó:

```
values[HPG] giữ nguyên
=
27.5


marks[HPG] giữ nguyên
=
55
```

Ghi nhận:

```
PRICE_STALE(29/04/2024, HPG)
```

MWG

```
values[MWG]
=
23.020833 × 44 / 40
≈
25.322917
```

Sau đó:

```
marks[MWG] = 44
```

SSI

```
values[SSI]
=
18.541667 × 27 / 30
=
16.6875
```

Sau đó:

```
marks[SSI] = 27
```

VCB

Lần BUY đầu tiên kỳ 3 đã lưu:

```
marks[VCB] = 100
```

Tại `end_date`:

```
current_price_VCB = 105
```

Do đó:

```
values[VCB]
```

```
=
13.75 × 105 / 100
=
14.4375
```

Sau đó:

```
marks[VCB] = 105
```

NAV CUỐI

```
NAV_end
=
29.90625
+
27.5
+
25.322917
+
16.6875
+
14.4375
```

suy ra:

```
NAV_end
≈
113.854167 triệu đồng
```

Tại `end_date` :

```
không tạo Target mới

không tính delta_weight

không BUY

không SELL

không rebalance

không tạo turnover mới

không tạo holdings sau rebalance mới
```

Chỉ thực hiện:

```
revalue
↓
PRICE_STALE nếu thiếu giá
↓
tính NAV cuối
↓
append NAV
```

NAV XUYÊN SUỐT 3 KỲ

`nav_<strategy>.csv`:

date	nav
01/02/2024	100.000000
01/03/2024	107.500000
01/04/2024	110.000000
29/04/2024	113.854167

Lợi suất có thể được suy từ chuỗi NAV:

```
01/02 → 01/03:

107.5 / 100 - 1
=
+7.50%


01/03 → 01/04:

110 / 107.5 - 1
≈
+2.3256%


01/04 → 29/04:

113.854167 / 110 - 1
≈
+3.5038%
```

Engine không tính return hằng ngày trong holding period.

TỔNG HỢP CÁC TRƯỜNG HỢP ĐÃ MINH HỌA

Trường hợp	Kỳ	Kết quả
`N_candidates = 11`	Cả 3 kỳ	`ceil(30% × 11) = 4`
Equal-weight allocation của Target minh họa	Cả 3 kỳ	4 mã × 25%
Financial PIT	Kỳ 1 → 2	FPT chuyển FY2022 → FY2023 khi `usable_from_date` cho phép
`FINANCIAL_NOT_AVAILABLE`	Kỳ 1	CTG không vào Financial Eligible Pool
Portfolio ban đầu 100% Cash	Kỳ 1	`revalue()` không có ticker để xử lý
BUY thành công	Kỳ 1	FPT, HPG, VCB
BUY_FAILED do giá trần	Kỳ 1	MWG
BUY_FAILED giữ Cash	Kỳ 1	25 triệu còn ở Cash
Weight drift	Kỳ 2	FPT từ 25% → khoảng 27.907%
`PRICE_STALE`	Kỳ 2	HPG thiếu giá
Giữ last valid mark	Kỳ 2	`marks[HPG] = 50`
SELL một phần	Kỳ 2	FPT có `intended_sell_value = 3.125`
SELL toàn bộ thành công	Kỳ 2	VCB bị xóa khỏi `values/marks`
SELL_FAILED do thiếu price	Kỳ 2	HPG
SELL_FAILED làm thiếu Cash	Kỳ 2	Không nhận được 25 triệu từ HPG
Proportional scaling	Kỳ 2	MWG/VNM/SSI cùng scale ≈ 0.689922
Giá xuất hiện lại sau stale	Kỳ 3	HPG được revalue từ mark 50 → 55
Không carry action cũ	Kỳ 3	HPG từ SELL_FAILED → HOLD
HOLD	Kỳ 3	HPG có `delta_weight = 0`
SELL toàn bộ thành công	Kỳ 3	VNM
SELL_FAILED do giá sàn	Kỳ 3	SSI
Nhiều BUY cùng scale	Kỳ 3	FPT/MWG/VCB cùng scale = 0.5
Position mới	Kỳ 3	`values[VCB]=13.75`, `marks[VCB]=current_price tại execution_date`

Portfolio không reset	Cả 3 kỳ	`cash`, `values`, `marks` được carry
Missing price tại `end_date`	Cuối kỳ 3	HPG tiếp tục `PRICE_STALE`
`end_date` không rebalance	Cuối kỳ 3	Chỉ revalue + append NAV
Failed/HOLD không tạo turnover	Cả 3 kỳ	`executed = 0` nên không đi vào `traded`

TÓM TẮT VÒNG LẶP

```
KỲ 1

Bước 1
→ 11 candidates
→ top 4
→ FPT / HPG / MWG / VCB
→ mỗi mã 25%

Bước 2
→ BUY FPT
→ BUY HPG
→ MWG BUY_FAILED
→ BUY VCB

Bước 3
→ giữ FPT + HPG + VCB + Cash

Bước 4
→ sang kỳ 2
        ↓

KỲ 2

Bước 1
→ 11 candidates
→ FPT / MWG / VNM / SSI

Bước 2
→ revalue
→ FPT weight drift
→ HPG PRICE_STALE
→ FPT SELL một phần
→ HPG SELL_FAILED
→ VCB SELL toàn bộ
→ MWG / VNM / SSI BUY bị SCALED
```

```
Bước 3
→ giữ Actual Portfolio

Bước 4
→ sang kỳ 3
        ↓

KỲ 3

Bước 1
→ 11 candidates
→ FPT / HPG / MWG / VCB

Bước 2
→ revalue
→ HPG có giá trở lại
→ HPG HOLD
→ VNM SELL thành công
→ SSI SELL_FAILED
→ FPT / MWG / VCB BUY bị SCALED
→ VCB là position mới
→ marks[VCB] lấy current_price tại execution_date

Bước 3
→ giữ Actual Portfolio tới end_date

Bước 4
→ không còn kỳ tiếp theo
→ revalue tại end_date
→ HPG PRICE_STALE
→ append NAV cuối
→ kết thúc segment
```

---

16. SCHEMA CANONICAL M3-A THEO OUTPUT GITHUB

Mục này định nghĩa các canonical output của M3-A theo implementation GitHub hiện hành.

Ý nghĩa của từng field được ghi trực tiếp trong schema để tránh tạo thêm bảng mô tả lặp lại.

16.1. `targets_<strategy>.csv`

Ý nghĩa:

> Strategy muốn nắm những mã nào tại mỗi `decision_date`.

Schema:

```
decision_date
→ ngày strategy tạo Target Portfolio bằng thông tin hợp lệ tại thời điểm đó

ticker
→ mã cổ phiếu được chọn vào Target Portfolio
→ có thể để trống nếu kỳ đó chỉ ghi status và không có pick

rank
→ thứ tự ticker trong danh sách được chọn
→ bắt đầu từ 1

target_weight
→ tỷ trọng mục tiêu mà strategy muốn phân bổ cho ticker

candidates
→ tổng số ticker trong Candidate Universe trước bước chọn cuối cùng
→ không phải số ticker cuối cùng được chọn

status
→ trạng thái tạo target tại kỳ đó
→ ví dụ:
   OK
   UNIVERSE_TOO_SMALL
```

Ví dụ:

```
candidates = 20
top_share = 30%
→ n_select = 6
```

nên:

```
candidates
≠
số ticker cuối cùng được chọn
```

16.2. `trade_ledger_<strategy>.csv`

Ý nghĩa:

> Engine muốn BUY/SELL gì tại `execution_date` và thực tế đã thực hiện được bao nhiêu.

Schema:

```
date
→ execution_date của kỳ rebalance

ticker
→ mã cổ phiếu mà action áp dụng
```

```
side
→ hướng action
→ BUY / SELL / HOLD

pre_weight
→ tỷ trọng thực tế của ticker
  sau khi Current Portfolio được revalue
  nhưng trước rebalance

target_weight
→ tỷ trọng Target Portfolio yêu cầu cho ticker
→ nếu ticker không còn trong target thì bằng 0

intended
→ giá trị giao dịch engine muốn thực hiện
  trước khi xét failed trade hoặc scaling

executed
→ giá trị giao dịch thực tế đã được thực hiện
→ failed trade thì bằng 0

scale
→ hệ số scaling của BUY executable khi Cash không đủ
→ tối đa bằng 1

status
→ kết quả thực thi action
→ EXECUTED
→ BUY_FAILED
→ SELL_FAILED
→ SCALED
→ HOLD
```

Quan hệ:

```
pre_weight
+
target_weight
↓
xác định phần cần rebalance
↓
intended
↓
tradability + available Cash
↓
executed
```

Lưu ý:

```
scale = 1.0
```

không tự động có nghĩa giao dịch thành công.

Ví dụ:

```
side = BUY
executed = 0
scale = 1.0
status = BUY_FAILED
```

Do đó phải đọc:

```
scale
+
executed
+
status
```

cùng nhau.

16.3. `holdings_<strategy>.csv`

Ý nghĩa:

> Sau rebalance, Portfolio thực tế đang nắm giữ những gì.

Schema:

```
date
→ execution_date mà holdings được ghi lại
→ trạng thái sau khi rebalance kỳ đó hoàn tất

ticker
→ tài sản thực tế đang nằm trong Portfolio
→ có thể là cổ phiếu hoặc:
   CASH

weight
→ tỷ trọng thực tế của ticker hoặc Cash trong NAV
```

Đối với cổ phiếu:

```
weight
=
values[ticker] / NAV
```

Đối với Cash:

```
weight
=
```

```
cash / NAV
```

Cần phân biệt:

```
targets_<strategy>.csv
→ strategy muốn nắm gì

holdings_<strategy>.csv
→ Portfolio thực tế đang nắm gì
```

Hai file có thể khác nhau do:

```
BUY_FAILED
SELL_FAILED
SCALED
```

Canonical holdings hiện tại chỉ persist:

```
date
ticker
weight
```

không persist:

```
position_value
```

16.4. `turnover_<strategy>.csv`

Ý nghĩa:

> Mức độ giao dịch thực tế của Portfolio tại mỗi kỳ rebalance.

Schema:

```
date
→ execution_date của kỳ rebalance

turnover
→ tỷ lệ giá trị BUY + SELL thực sự executed
  so với pre_trade_NAV
```

Theo contract hiện tại:

```
traded
=
executed BUY
+
```

```
executed SELL
```

và:

```
turnover
=
traded
/
(2 × pre_trade_NAV)
```

Do đó:

```
BUY_FAILED
SELL_FAILED
```

có:

```
executed = 0
```

nên không làm tăng turnover.

16.5. `nav_<strategy>.csv`

Ý nghĩa:

> Giá trị toàn bộ Portfolio của strategy theo thời gian.

Schema:

```
date
→ ngày Portfolio được valuation
→ thường gồm:
   execution_date
   và valuation cuối tại end_date

nav
→ tổng giá trị Portfolio tại ngày đó
```

Trong engine:

```
NAV
=
cash
+
Σ values[ticker]
```

Tại `execution_date` :

```
Current Portfolio
↓
revalue
↓
ghi NAV
↓
rebalance
```

Nếu một ticker không có giá hợp lệ khi revalue:

```
PRICE_STALE
```

thì engine giữ mark trước đó cho vị thế đó.

16.6. `benchmark_vnindex.csv`

Ý nghĩa:

> Đường NAV chuẩn hóa của VNINDEX dùng để so sánh với strategy.

Schema:

```
date
→ valuation date dùng để so sánh với strategy

nav
→ VNINDEX close đã được chuẩn hóa về NAV
  với ngày đầu tiên bằng 1
```

Implementation:

```
benchmark_nav(date)
=
VNINDEX_close(date)
/
VNINDEX_close(first_date)
```

Do đó:

```
benchmark NAV tại ngày đầu
=
1
```

File này lưu normalized NAV, không phải trực tiếp giá trị điểm VNINDEX gốc.

16.7. `metrics.json`

Ý nghĩa:

> Tổng hợp performance/execution metrics của từng strategy và benchmark.

Schema chính cho mỗi strategy:

```
periods
→ số khoảng return giữa các NAV liên tiếp
→ không nhất thiết bằng số decision_date

total_return
→ tổng lợi suất từ NAV đầu đến NAV cuối

cagr
→ tốc độ tăng trưởng kép năm hóa

volatility_annual
→ volatility của period return
  được annualize theo sqrt(12)

max_drawdown
→ mức sụt giảm NAV lớn nhất
  so với peak trước đó

avg_turnover
→ turnover trung bình của các kỳ rebalance

avg_cash_after_trading
→ tỷ trọng CASH trung bình
  sau các lần rebalance

failed_buys
→ số ledger row có:
   status = BUY_FAILED

failed_sells
→ số ledger row có:
   status = SELL_FAILED

scaled_buys
→ số BUY row có:
   status = SCALED

price_stale
→ số lần xuất hiện cặp:
   (date, ticker)
   mà engine không có giá mới hợp lệ khi revalue
```

Nếu segment cho phép Sharpe:

```
sharpe_rf0
→ Sharpe ratio với risk-free rate = 0
```

Ngoài từng strategy, `metrics.json` còn có:

```
vnindex
→ metrics tính trên benchmark VNINDEX

adj_glitches_repaired
→ số adjusted-price glitches
  được PriceBook repair trong run
```

16.8. `manifest.json`

Ý nghĩa:

> Metadata nhận diện, mô tả lần chạy M3 và ghim nguồn gốc (provenance) để bảo đảm khả năng tái lập (reproducibility) theo Mục 5.3 và Mục 6.

Schema:

```
config
→ đường dẫn config được sử dụng cho run

config_sha256
→ SHA-256 của nội dung config
→ dùng để xác định chính xác config đã chạy

git_commit
→ mã băm SHA git commit tại thời điểm thực thi run
→ bảo đảm tính tái lập của thuật toán và mã nguồn

status
→ trạng thái config tại thời điểm run
→ ví dụ:
   PROPOSED
   LOCKED

segment
→ segment đang chạy
→ development
→ holdout
→ final

run_id
→ định danh duy nhất của lần chạy
→ gồm UTC timestamp + phần hash của config

targets_only
→ true:
   chỉ tạo Target Portfolio

→ false:
```

```
   chạy full backtest

counts
→ thống kê target-generation theo từng strategy

inputs_sha256
→ mã băm SHA-256 của toàn bộ runtime inputs bắt buộc để đảm bảo tính tái lập:
   terminal_sha256: hash của data/web/delta-intelligence-v4/terminal.sqlite
   indicators_sha256: hash của indicators.sqlite
   availability_sha256: hash của availability.csv
   b_pit_sha256: hash của B-PIT artifact
   c8_sha256: hash của C8 canonical bundle (benchmark, calendar, limits/prices)
```

Bên trong:

```
counts
└── <strategy>
    ├── decisions
    │   → tổng số decision_date được xử lý
    │
    └── too_small
        → số kỳ có:
          status = UNIVERSE_TOO_SMALL

inputs_sha256
├── terminal_sha256
├── indicators_sha256
├── availability_sha256
├── b_pit_sha256
└── c8_sha256
```

Canonical manifest đầy đủ gồm:

```
config
config_sha256
git_commit
status
segment
run_id
targets_only
counts
inputs_sha256
```

Đồng bộ provenance: Theo yêu cầu của Mục 5.3 và Mục 6, manifest bắt buộc phải ghim provenance gồm commit hash và toàn bộ runtime inputs. Trước khi chuyển sang trạng thái `LOCKED` và thực thi official run, script `scripts/run_m3.py` phải gọi hàm `inputs.sha256()` đã có sẵn để ghi nhận đầy đủ các mã băm này vào `manifest.json`.

16.9. Quan hệ giữa các canonical output

```
Strategy
↓
targets_<strategy>.csv
→ strategy muốn nắm gì
↓
Execution Engine
↓
trade_ledger_<strategy>.csv
→ thực tế BUY/SELL gì
↓
holdings_<strategy>.csv
→ thực tế đang nắm gì
↓
turnover_<strategy>.csv
→ giao dịch nhiều hay ít
↓
nav_<strategy>.csv
→ danh mục có giá trị bao nhiêu
↓
metrics.json
→ kết quả tổng hợp

VNINDEX
↓
benchmark_vnindex.csv
↓
metrics.json

Run metadata
↓
manifest.json
```

16.10. `universe_audit_<strategy>.csv`

Ý nghĩa:

> Tại mỗi `decision_date`, ticker nào được xét, có vào Candidate Universe không, và nếu không thì vì sao.

Schema:

```
decision_date
ticker
market_cluster
market_eligible          (true/false)
financial_year           (selected_fiscal_year hoặc trống)
financial_cluster
financial_eligible       (true/false hoặc trống với strategy không dùng Financial)
in_candidate_universe    (true/false)
reason                   (một primary reason, trống nếu được chọn vào Candidate Universe)
flags                    (ví dụ FC_MISSING:roe)
```

`reason` dùng đúng tập lý do của Mục 3: `MARKET_ILLIQUID`, `MARKET_NOT_IN_LEADING_CLUSTER`, `CLUSTER_PROFILE_AMBIGUOUS`, `FINANCIAL_SECTOR_NO_B`, `FINANCIAL_NOT_AVAILABLE`, `FINANCIAL_STALE`, `FINANCIAL_STATE_MISSING`, `FINANCIAL_NOT_IN_ALLOWED_CLUSTER`, `FINANCIAL_CRITERIA_MISSING`.

Kiểm tra: số dòng `in_candidate_universe = true` tại mỗi kỳ phải bằng `candidates` trong `targets_<strategy>.csv`.

Equal-weight và Momentum-only có thể chỉ ghi `MARKET_ILLIQUID` / thiếu `mom_126`.

---

17. BENCHMARK — VNINDEX

17.1. Vai trò

VNINDEX là benchmark chính thức hiện hành. Market-only, Financial-only, Equal-weight và Momentum-only là research controls, không phải benchmark.

17.2. Căn thời gian

VNINDEX phải được căn đúng các evaluation/execution dates của từng segment, không tạo gap legacy và không dùng ngày tương lai để thay cho quan sát thiếu.

17.3. Return basis — Q11

Main canonical M3 sử dụng:

```
price basis = adj_close
transaction cost = 0
slippage = 0
cash return = 0
```

`adj_close` được mô tả là adjusted total-return proxy theo input hiện hành.

VNINDEX là price index. Do đó mọi bảng/biểu so sánh phải ghi rõ basis mismatch; không mô tả hai chuỗi là hoàn toàn tương đương về return basis.

Secondary price-only

Secondary price-only scenario được giữ như một validation/sensitivity concept nhưng hiện:

```
SECONDARY_PRICE_ONLY = NOT_READY
```

Lý do: project hiện chưa có corporate-action dataset/readiness đủ để tái dựng canonical price-only series một cách reproducible.

Do đó:

- không chạy price-only
- không tự coi `raw_close` là canonical price-only
- không block main `adj_close` backtest

Khi sau này có corporate-action readiness, price-only phải mở qua readiness gate riêng.

17.4. Metrics

Development/Holdout tối thiểu có thể báo cáo:

```
Total/Cumulative Return
CAGR nếu hợp lệ
Annualized Volatility
Maximum Drawdown
Turnover (strategy only)
Cash ratio (strategy only)
failed execution counts (strategy only)
```

Các metric bổ sung chỉ dùng khi contract đủ điều kiện. Sharpe không dùng trong Development/Holdout; Final Evaluation mới được phép báo cáo Sharpe.

17.5. Final Evaluation

Chỉ mở khi dữ liệu sau `2025-12-31` pass readiness + unseen-status audit và exact window đã khóa trước khi xem performance.

18. SAU KHI CHẠY XONG 4 BƯỚC MỚI TÍNH PERFORMANCE

Đầu vào canonical:

```
nav_<strategy>.csv
turnover_<strategy>.csv
holdings_<strategy>.csv
trade_ledger_<strategy>.csv
benchmark_vnindex.csv
```

Runner/reporting tạo `metrics.json` với tối thiểu:

```
periods
total_return
cagr
volatility_annual
max_drawdown
avg_turnover
avg_cash_after_trading
failed_buys
failed_sells
scaled_buys
```

```
price_stale
```

Sharpe chỉ ở Final Evaluation.

Kịch bản chính có transaction cost = 0 và slippage = 0,
do đó gross return và net return của base scenario là tương đương.

---

19. CHECKLIST TOÀN BỘ M3-A THEO 4 BƯỚC

19.1. Bước 1 — Chọn cổ phiếu

```
[ ] Market input/version/hash validated
[ ] B-PIT built + audited + pinned
[ ] Financial input/version/hash validated
[ ] Financial runtime PIT = usable filter → max(fiscal_year) frozen
[ ] Q-date availability rule frozen
[ ] Financial stale_months = 18 frozen
[ ] ILLIQUID exclusion frozen
[ ] Market leading selector = median mom_126; tie median mdd_126 frozen
[ ] Financial allowed clusters = not_weakest; order ROE then D/E frozen
[ ] MC1 = mom_126 / động lượng 6 tháng frozen
[ ] MC2 = downside_vol_63 / biến động giảm 63 phiên frozen
[ ] MC3 = liquidity_21 / thanh khoản 21 phiên frozen
[ ] FC1 = ROE frozen
[ ] FC2 = F-score frozen
[ ] FC3 = Z-score frozen
[ ] Financial missing rule 0.5 + all-missing exclusion frozen
[ ] S_market formula frozen
[ ] S_fin formula frozen
[ ] Combined = 0.5 × S_market + 0.5 × S_fin frozen
[ ] tie = liquidity_21 desc → ticker asc frozen
[ ] n_min = 10 frozen
[ ] targets_<strategy>.csv created
```

19.2. Bước 2 — Giả lập mua/bán

```
[ ] execution_date > decision_date
[ ] execution_date phải nằm trong boundary của segment để được execution
[ ] Target boundary vẫn persist với NOT_EXECUTED_SEGMENT_END
[ ] Current Portfolio revalued
[ ] Current-vs-Target calculated
[ ] SELL executed before BUY
[ ] volume=0 blocks BUY+SELL
[ ] raw_close=ceiling blocks BUY
[ ] raw_close=floor blocks SELL
[ ] BUY_FAILED / SELL_FAILED / SCALED handled
[ ] Q17 adjusted-price repair test PASS
```

[ ] trade_ledger_<strategy>.csv created
[ ] holdings_<strategy>.csv đúng schema date + ticker + weight
[ ] CASH được biểu diễn như một row trong holdings
[ ] turnover_<strategy>.csv uses / (2 × pre_trade_NAV)

19.3. Bước 3 — Giữ và tính lời/lỗ

[ ] portfolio state carried, not reset
[ ] missing valuation price uses PRICE_STALE / stale mark
[ ] không suy missing price thành delisting
[ ] price_stale counted
[ ] NAV revalued at next execution/end date
[ ] nav_<strategy>.csv created
[ ] valuation_end_date không vượt segment_end

19.4. Bước 4 — Tái cân bằng

[ ] next Target recomputed with same frozen rules
[ ] no old action carried forward
[ ] weight drift enters next Current Portfolio
[ ] loop continues to segment end

19.5. Segment boundary validation

[ ] decision_date trong segment vẫn tạo/lưu Target
[ ] execution_date > segment_end → NOT_EXECUTED_SEGMENT_END
[ ] boundary Target không tạo trade/turnover/holding change
[ ] Development không sử dụng execution/valuation price sau 31/12/2023
[ ] Holdout không sử dụng execution/valuation price sau 31/12/2025
[ ] segment trước không sử dụng dữ liệu giá của segment kế tiếp để hoàn tất return

19.6. Return/benchmark validation

[ ] main price basis = adj_close
[ ] VNINDEX documented as price index
[ ] basis mismatch disclosed
[ ] secondary price-only = NOT_READY và không block main
[ ] main transaction cost = 0
[ ] main slippage = 0
[ ] cost sensitivity không nằm trong main run

19.7. Reproducibility

[ ] benchmark_vnindex.csv created

```
[ ] metrics.json created
[ ] manifest.json được tạo đúng schema GitHub hiện hành
[ ] config = LOCKED và hash pinned trước official run
```

PHẦN B — M3-B: DASHBOARD / PRODUCT LAYER

M3-B đưa các kết quả đã được upstream tạo và xác minh lên
web application hiện hành của repository.

Kiến trúc hiện hành:

web/frontend
→ Next.js frontend

web/backend
→ FastAPI read-only backend

M3-B là presentation/product layer, không phải research engine.

Các nguyên tắc bắt buộc:

- chỉ đọc versioned/sealed research artifacts;
- không fit lại Market/Financial clustering;
- không recompute research features/metrics nếu upstream đã cung cấp;
- không tự thực hiện PIT alignment riêng;
- không forward-fill hoặc biến missing thành 0;
- không biến cluster label thành khuyến nghị mua/bán;
- nếu hiển thị M3-A, phải truy được run/config/input/provenance tương ứng;
- frontend/backend chỉ trình bày kết quả, không được dẫn dắt methodology.

Chi tiết nhiệm vụ triển khai M3-B được giữ tại tài liệu M3-B riêng.
Master plan này chỉ giữ phạm vi, dependency và acceptance boundary.

PHẦN C — M3-C: BÁO CÁO KỸ THUẬT VÀ NOTEBOOK TÁI LẬP KẾT QUẢ

20. MỤC TIÊU M3-C

M3-C bám đúng yêu cầu:

> **Báo cáo kỹ thuật + notebook tái lập được kết quả.**

M3-C **không tạo phương pháp mới** và **không làm lại vai trò của M3-A hoặc M3-B**.

M3-C phải thực hiện ba việc chính:

1. **Báo cáo kỹ thuật** toàn bộ kết quả nghiên cứu M3, trọng tâm là kết quả M3-A gồm strategy chính Combined, bốn đối chứng Market-only, Financial-only, Equal-weight, Momentum-only và benchmark chính thức VNINDEX;
2. cung cấp các **notebook tái lập kết quả chính**, để từ đúng đầu vào, cấu hình và mã nguồn đã khóa có thể tạo lại các bảng, chỉ số và biểu đồ quan trọng dùng trong báo cáo;
3. ghi lại **kết quả đánh giá cuối** sau khi M3-A đã chạy, khóa và niêm phong kết quả đó.

M3-C có thể mô tả ngắn kiến trúc M3-B trong báo cáo kỹ thuật, nhưng:

- không tạo lại gói dữ liệu dashboard;
- không kiểm thử lại API/frontend thay cho M3-B;
- không cần notebook riêng cho dashboard;
- mọi xác minh nguồn → gói → API → giao diện thuộc trách nhiệm hoàn thành của M3-B.

Khái niệm **tái lập được kết quả** trong M3-C nghĩa là:

```
Đúng đầu vào đã khóa
+
Đúng cấu hình / quy tắc đã khóa
+
Đúng phiên bản mã nguồn
        ↓
Chạy notebook hoặc mô-đun chính thức
        ↓
Tạo lại được các bảng / chỉ số / biểu đồ chính
        ↓
Khớp với kết quả đã được M3-A niêm phong
```

Notebook không được viết một phiên bản business logic thứ hai chỉ để cho ra cùng kết quả. Nếu logic đã nằm trong mô-đun Python được kiểm thử, notebook phải gọi lại mô-đun đó hoặc đọc output chính thức rồi tái tạo phần trình bày/đối chiếu.

---

21. KẾ HOẠCH NOTEBOOK M3-C

M3-C giữ 4 notebook chính:

```
M3/notebooks/
├── 01_Du_Lieu_Va_PIT.ipynb
├── 02_Tai_Lap_Lua_Chon_Danh_Muc.ipynb
├── 03_Tai_Lap_Backtest_Va_Benchmark.ipynb
└── 04_Bao_Cao_Danh_Gia_Cuoi.ipynb
```

Notebook **không viết lại business logic**. Nó đọc sealed run hoặc gọi đúng module chính thức với cùng config/hash để kiểm chứng.

Bộ output canonical được M3-C phép phụ thuộc:

```
targets_<strategy>.csv
trade_ledger_<strategy>.csv
holdings_<strategy>.csv
turnover_<strategy>.csv
nav_<strategy>.csv
benchmark_vnindex.csv
metrics.json
manifest.json
```

22. M3-C NHIỆM VỤ 01 — NOTEBOOK DỮ LIỆU VÀ PIT

Mục tiêu

Chứng minh run dùng đúng input/version/hash và PIT contract mà **không cần** `aligned_market_financial_snapshots.csv`.

Đầu vào

```
manifest.json
config đã hash
Market/Financial handoff hoặc database input đã pin
availability/PIT artifact đã pin
targets_<strategy>.csv
```

Cách làm

1. Đọc `manifest.json` để xác định run_id, segment, config, config hash, status, targets_only và counts.
2. Đọc config và code commit.
3. Với một mẫu `decision_date`, gọi **chính module PIT/strategy chính thức** hoặc truy nguyên cùng input để kiểm tra `usable_from_date <= decision_date`; không viết rule PIT thứ hai trong notebook.
4. Đối chiếu target tái tạo với `targets_<strategy>.csv`.
5. Báo PASS/FAIL và mọi discrepancy.

Không tạo một alignment CSV canonical mới.

23. M3-C NHIỆM VỤ 02 — NOTEBOOK TÁI LẬP LỰA CHỌN DANH MỤC

Mục tiêu

Tái lập đúng logic tạo Target Portfolio của từng strategy bằng module chính thức và đối chiếu `targets_<strategy>.csv`.

Đầu vào

```
manifest.json
config
inputs đã pin
targets_<strategy>.csv
```

Đầu ra notebook

- một số bảng minh họa internal computation trong notebook;
- đối chiếu `decision_date/ticker/rank/target_weight/candidates/status` với `targets_<strategy>.csv`;
- không persist `combined_candidates.csv`, `decision_features.csv`, `combined_ranking.csv` thành canonical artifact.

24. M3-C NHIỆM VỤ 03 — NOTEBOOK TÁI LẬP BACKTEST VÀ BENCHMARK

Đầu vào

```
trade_ledger_<strategy>.csv
holdings_<strategy>.csv
turnover_<strategy>.csv
nav_<strategy>.csv
benchmark_vnindex.csv
metrics.json
manifest.json
```

Kiểm tra bắt buộc

- `holdings_<strategy>.csv` có đúng schema date/ticker/weight;
- `CASH` được biểu diễn đúng như một row khi phù hợp;
- tổng weight của holdings tại mỗi snapshot hợp lệ theo tolerance;
- failed trade có `executed=0` đúng semantics;
- `SCALED` khớp `scale`;
- turnover khớp `(executed BUY + executed SELL)/(2 × pre_trade_NAV)` khi tái tính từ ledger/NAV;
- NAV curve khớp output sealed;
- metrics tái tính từ NAV/turnover/holdings/ledger khớp `metrics.json` trong tolerance;

- VNINDEX dùng exact segment/window;
- Development/Holdout không tính Sharpe.

25. M3-C NHIỆM VỤ 04 — NOTEBOOK BÁO CÁO ĐÁNH GIÁ CUỐI

Chỉ chạy sau khi Final Evaluation đã sealed. Notebook đọc đúng bộ output canonical của run Final, tái tạo bảng/biểu đồ báo cáo và Sharpe nếu segment Final được phép. Không thay config, không tune lại và không mở một output intermediate legacy để thay thế canonical run.

---

26. RANH GIỚI TRÁCH NHIỆM CỦA M3-C

Để tránh lặp việc, giữ ranh giới sau:

```
M3-A
→ tạo và niêm phong kết quả nghiên cứu / backtest

M3-B
→ tạo và kiểm thử dashboard
→ chịu trách nhiệm truy nguyên nguồn → gói → API → giao diện

M3-C
→ tái lập các kết quả chính cần cho báo cáo
→ đối chiếu với artifact M3-A
→ viết báo cáo kỹ thuật
→ viết hướng dẫn tái lập
```

M3-C không làm lại:

- xác minh chính thức Market/Financial handoff thay M3-A;
- lựa chọn methodology mới;
- chạy lại clustering để chọn K mới;
- tạo Candidate / ranking bằng một logic khác M3-A;
- xây lại dashboard package;
- kiểm thử API/frontend thay M3-B;
- mở lại dữ liệu đánh giá cuối để tuning strategy.

---

27. ĐẦU RA BÀN GIAO M3-C

M3-C tối thiểu bàn giao:

```
M3/notebooks/
```

```
├── 01_Du_Lieu_Va_PIT.ipynb
├── 02_Tai_Lap_Lua_Chon_Danh_Muc.ipynb
├── 03_Tai_Lap_Backtest_Va_Benchmark.ipynb
└── 04_Bao_Cao_Danh_Gia_Cuoi.ipynb

M3/HUONG_DAN_THUC_HIEN_M3.md

M3/reports/
└── BAO_CAO_KY_THUAT_M3.md
```

Ngoài ra phải có bằng chứng đối chiếu cho biết các bảng / chỉ số / biểu đồ trong notebook và báo cáo được sinh từ version/hash nào của input và code.

Không bắt buộc tạo một bản sao mới của toàn bộ artifact M3-A/M3-B trong thư mục M3-C.

---

28. ĐIỀU KIỆN HOÀN THÀNH NỘI BỘ M3-C

M3-C chỉ được coi là hoàn tất khi:

- 4 notebook chính chạy được với đầu vào đã version hóa;
- mỗi notebook ghi rõ input, code version và output;
- Notebook 01 chứng minh được PIT/no-look-ahead từ manifest + input đã pin + module chính thức và đối chiếu `targets_<strategy>.csv`;
- Notebook 02 tái lập được Candidate → 6 tiêu chí → ranking → top 30% và khớp `targets_<strategy>.csv`;
- Notebook 03 tái lập được `ledger/holdings/turnover/nav/benchmark/metrics` theo từng đoạn;
- Notebook 04 trình bày lại được đánh giá cuối từ output đã niêm phong mà không thay strategy;
- Báo cáo kỹ thuật lấy số liệu từ tệp / notebook, không nhập tay;
- `HUONG_DAN_THUC_HIEN_M3.md` chỉ rõ thứ tự, command và điều kiện dừng;
- mọi số liệu chính truy ngược được về input, config, code version và artifact nguồn;
- không có notebook riêng lặp lại nhiệm vụ xây / kiểm thử dashboard của M3-B.

29. BÁO CÁO KỸ THUẬT M3

Báo cáo kỹ thuật là đầu ra chính của yêu cầu **"Báo cáo kỹ thuật"** trong M3-C.

Báo cáo tối thiểu nên có:

1. Mục tiêu nghiên cứu M3.
2. Nguồn dữ liệu và các phiên bản chính được sử dụng.
3. Kiến trúc hai hệ phân cụm độc lập.
4. Cách căn chỉnh Market–Financial theo thời gian/PIT.
5. Quy tắc tạo Candidate Universe.
6. Ba tiêu chí Market và ba tiêu chí Financial.

7. Quy tắc xếp hạng Combined và chọn top 30%.
8. Cơ chế thực hiện giao dịch, tiền mặt và tái cân bằng.
9. Kết quả strategy chính Combined.
10. Kết quả bốn đối chứng nghiên cứu:

- Market-only;
- Financial-only;
- Equal-weight;
- Momentum-only.

11. Benchmark chính thức: VNINDEX.
12. Kết quả lịch sử Development và Holdout trình bày tách biệt.
13. Các chỉ số hiệu quả / rủi ro / turnover / cash / failed execution đã được duyệt.
14. Kết quả đánh giá cuối.
15. Giới hạn dữ liệu và giới hạn phương pháp.
16. Khả năng tái lập: input version, config version, code version, hash và notebook tương ứng.
17. Phần mô tả ngắn về M3-B và đường dẫn / bằng chứng sang kết quả dashboard nếu cần cho tài liệu tổng thể.

Báo cáo không được:

- nhập tay các con số mà không truy ngược được về tệp nguồn;
- thay phương pháp để làm kết quả đẹp hơn;
- biến nhãn phân cụm thành khuyến nghị đầu tư;
- trình bày dashboard như bằng chứng thay thế cho kết quả nghiên cứu gốc.

---

30. HƯỚNG DẪN TÁI LẬP

Mục tiêu là để một người mới có thể đi từ:

```
Repository + đúng commit
        +
Runtime inputs đã version hóa
        +
Config đã khóa
        +
Module chính thức
        ↓
Tái lập / kiểm chứng kết quả M3
        ↓
Đối chiếu với canonical artifacts
```

Không viết lại business logic M3 trong tài liệu hoặc notebook.

Nếu logic đã tồn tại trong:

```
src/delta_t1/m3/
scripts/run_m3.py
```

thì phải sử dụng lại module/command chính thức đó.

30.1. Mỗi bước trong hướng dẫn phải ghi

Mỗi bước tối thiểu phải có:

- mục đích;
- input/tệp cần dùng;
- path thực tế;
- version/hash cần kiểm tra;
- config sử dụng;
- code commit;
- command hoặc module chính thức;
- output cần tạo hoặc kiểm tra;
- cách đối chiếu với canonical output;
- điều kiện PASS;
- điều kiện FAIL/DỪNG;
- bước tiếp theo.

Không ghi command cho notebook/module chưa tồn tại hoặc chưa được xác minh trên repository.

30.2. Luồng tái lập tổng quát

```
Kiểm tra repository / branch / commit / môi trường
        ↓
Kiểm tra config
        ↓
Kiểm tra manifest
        ↓
Kiểm tra 6 logical runtime inputs
        ↓
Kiểm tra version / hash / schema / coverage
        ↓
Notebook 01 — Dữ liệu và PIT
        ↓
Notebook 02 — Tái lập lựa chọn danh mục
        ↓
Notebook 03 — Tái lập backtest và benchmark
        ↓
Final Evaluation đã verified + sealed?
        |
```

```
          ├── CHƯA
          │      ↓
          │   Notebook 04 = NOT_READY
          │
          └── RỒI
                 ↓
          Notebook 04 — Báo cáo đánh giá cuối
                 ↓
Đối chiếu canonical artifacts
                 ↓
Hoàn thiện Báo cáo kỹ thuật M3
```

Notebook có nhiệm vụ:

```
đọc
tái lập
đối chiếu
trình bày
```

không tạo một implementation M3 thứ hai.

Nếu cần chạy lại toàn bộ M3-A, phải gọi đúng `scripts/run_m3.py` và các module chính thức.

30.3. Các trường hợp phải DỪNG

Phải DỪNG nếu xảy ra ít nhất một trường hợp:

- thiếu runtime input bắt buộc;
- path input không đúng;
- hash/version không khớp;
- schema thực tế không đáp ứng loader contract;
- coverage không đủ cho segment cần chạy;
- Market/Financial/PIT input chưa đạt readiness;
- code commit không xác định được;
- config cần cho official backtest chưa ở trạng thái `LOCKED`;
- owner decision đã khóa nhưng ADR/config/code tương ứng chưa được sync hoặc không khớp decision;
- canonical backtest artifact chưa tồn tại hoặc chưa được verify;
- notebook tái lập không khớp canonical output mà chưa giải thích được;
- VNINDEX không có đúng segment/window;
- Final window chưa được khóa;
- Final input chưa đạt readiness;
- Final run chưa sealed nhưng Notebook 04 được yêu cầu chạy.

Không sửa methodology/config chỉ để làm notebook khớp kết quả.

30.4. Kiểm tra repository và môi trường

Chạy tối thiểu:

```
git branch --show-current
git rev-parse HEAD
git status --short

.venv\Scripts\python.exe -m unittest discover -s tests -v
.venv\Scripts\python.exe -m compileall -q src tests scripts run.py
```

Ghi lại:

```
BRANCH
HEAD
WORKTREE STATUS
CONFIG PATH
CONFIG SHA256
```

Nếu local và remote diverge:

```
LOCAL_HEAD
REMOTE_HEAD
DIVERGENCE
```

Không tự reset local theo remote.

30.5. Command M3-A hiện được GitHub hỗ trợ

Runner chính thức:

```
scripts/run_m3.py
```

Khi config còn PROPOSED

Chỉ chạy targets-only:

```
python scripts/run_m3.py --segment development --targets-only
```

Targets-only dùng để kiểm tra:

```
PIT
Candidate Universe
candidate count
UNIVERSE_TOO_SMALL
ranking/picks
```

```
target_weight
status
```

Không tính return.

Không coi targets-only là official backtest.

Khi config đã `LOCKED`

Có thể chạy:

```
python scripts/run_m3.py --segment development
python scripts/run_m3.py --segment holdout
```

Official historical backtest không được chạy khi config vẫn `PROPOSED` .

30.6. Phân biệt output theo loại run

Targets-only

Tạo tối thiểu:

```
targets_<strategy>.csv
manifest.json
```

Không yêu cầu:

```
trade_ledger
holdings
turnover
nav
benchmark
metrics
```

Full backtest

Tạo:

```
targets_<strategy>.csv
trade_ledger_<strategy>.csv
holdings_<strategy>.csv
turnover_<strategy>.csv
nav_<strategy>.csv
benchmark_vnindex.csv
metrics.json
```

`manifest.json`

Hướng dẫn phải xác định loại run trước khi kiểm tra output.

---

31. THỨ TỰ THỰC HIỆN TOÀN DỰ ÁN

Mục này chỉ quy định **thứ tự thực hiện**. Chi tiết methodology, input contract, output contract và execution semantics được định nghĩa tại các mục tương ứng phía trên và không lặp lại tại đây.

Giai đoạn 0 — Sync decision đã khóa vào Git/config

Decision queue Q1–Q17/Q-date đã được owner khóa về methodology.

Trước official backtest có return còn phải, theo đúng thứ tự:

1. ghi decision vào `docs/DECISIONS.md` /ADR, gồm cả các điểm R1, R2, R7 đã được owner xác nhận (làm trước tiên vì `DECISIONS.md` đứng đầu thứ tự precedence ở Mục 1);
2. cập nhật plan/config/code theo các rule đã khóa (config theo Mục 36.4; code: `stale_months` đọc từ config, strict segment boundary, fail-closed trần/sàn, N_min sau `FINANCIAL_CRITERIA_MISSING`, persist `universe_audit`, manifest provenance);
3. build/audit/pin B-PIT;
4. chạy lại feasibility với B-PIT (Giai đoạn 3);
5. chuyển `status = LOCKED`;
6. pin config hash.

Không thay strategy rule dựa trên return sau khi đã khóa.

Giai đoạn 1 — Audit và pin runtime inputs

Xác minh toàn bộ runtime inputs theo **Mục 5 — Khung dữ liệu chung**:

- đúng path/version;
- đúng schema;
- đủ coverage cho segment;
- hash đã xác minh;
- provenance đã pin.

Nếu một input bắt buộc chưa PASS thì không chuyển sang official backtest.

Giai đoạn 2 — Hoàn thiện output writer và manifest

Trước official run phải bảo đảm:

- canonical output contract đã sẵn sàng;
- `holdings_<strategy>.csv` được tạo đúng schema GitHub hiện hành:

```
date
ticker
weight
```

- `manifest.json` được tạo đúng schema GitHub hiện hành;
- execution semantics và turnover contract đã được kiểm thử.

Giai đoạn 3 — Targets-only readiness

Khi input đã sẵn sàng nhưng config còn `PROPOSED`, chỉ chạy targets-only để kiểm tra:

- PIT;
- eligibility;
- Candidate Universe;
- ranking;
- picks;
- target weights;
- trạng thái như `UNIVERSE_TOO_SMALL`.

Trước khi LOCKED, chạy lại `scripts/m3_feasibility_variants.py` (chỉ đếm, không tính return) với B-PIT thay cho B v4 và báo cáo:

```
trung vị / min N_candidates theo segment
số tháng N_candidates < 10
tỷ lệ loại theo reason (từ universe_audit)
```

Các con số hiện có (Va: Development trung vị 26, 13/60 tháng < 10) đều đo trên B v4. Nếu số tháng < 10 tăng đáng kể với B-PIT, owner quyết định lại N_min/Q5 tại bước này, trước khi có bất kỳ con số return nào.

Targets-only không tính return và không được coi là official backtest.

Giai đoạn 4 — Historical backtest

Chỉ chạy khi:

```
config = LOCKED
+
runtime inputs PASS
+
writer/manifest PASS
+
tests PASS
```

Thứ tự:

```
Development
2019-01-31 → 2023-12-29
```

```
60 decision snapshots
        ↓
Verify + Seal
        ↓
Holdout
2024-01-31 → 2025-12-31
24 decision snapshots
        ↓
Verify + Seal
```

Holdout sử dụng đúng frozen config/engine của Development và không được dùng để tune strategy.

Giai đoạn 5 — Final Evaluation

Chỉ mở khi dữ liệu sau `2025-12-31` đạt readiness.

Trước khi xem Final performance:

1. audit Market/Financial/PIT/prices/execution/VNINDEX;
2. khóa exact Final start/end;
3. pin config/version/hash;
4. chạy bằng strategy rules và engine đã frozen;
5. verify;
6. seal;
7. mới thực hiện đánh giá Final theo contract đã khóa.

Không dùng Final result để quay lại chỉnh strategy.

Giai đoạn 6 — M3-B

Sau khi các artifact upstream cần thiết đã sẵn sàng, M3-B đọc các versioned/sealed artifacts để trình bày trên product/dashboard layer.

M3-B không fit model, không recompute research logic và không thay methodology.

Chi tiết xem tài liệu M3-B riêng.

Giai đoạn 7 — M3-C

Sau khi các canonical run cần thiết đã tồn tại:

- tái lập dữ liệu/PIT;
- tái lập lựa chọn danh mục;
- tái lập backtest/benchmark;
- lập báo cáo đánh giá cuối;
- đối chiếu kết quả với sealed canonical artifacts.

Notebook không tạo một implementation business logic thứ hai.

Luồng tổng quát:

```
LOCK DECISIONS
      ↓
AUDIT + PIN INPUTS
      ↓
WRITER / MANIFEST READY
      ↓
TARGETS-ONLY
      ↓
DEVELOPMENT
      ↓
HOLDOUT
      ↓
FINAL EVALUATION
      ↓
M3-B + M3-C
```

---

32. CÁC ĐIỀU TUYỆT ĐỐI KHÔNG LÀM

Các nguyên tắc dưới đây là ranh giới bất biến của M3.

Không gộp Market và Financial thành một mô hình phân cụm mới trong M3.
Market Clustering và Financial Clustering là hai hệ độc lập; M3 chỉ kết hợp chúng tại tầng strategy/portfolio theo rule đã khóa.

Không dùng thông tin tương lai.
Không dùng Financial data chưa usable tại `decision_date` và không dùng giá sau `decision_date` để tính signal/ranking tại ngày đó.

Không dùng `period_end` thay cho Financial availability/PIT timing.
Financial state chỉ được sử dụng khi `usable_from_date <= decision_date`.

Không thay dữ liệu lịch sử bằng trạng thái hiện tại.
Market state và Financial state của mỗi `decision_date` phải được lấy từ dữ liệu tương ứng với thời điểm đó.

Không thay methodology/strategy sau khi xem performance.
Không thay cluster authority, eligibility, tiêu chí MC/FC, trọng số, top-share, tie rule, `N_min`, execution rule hoặc return basis chỉ vì kết quả backtest không thuận lợi.

Không tạo hoặc che dữ liệu để làm kết quả đẹp hơn.
Không forward-fill/bịa observation, không biến missing thành 0, không xóa kết quả bất lợi và không sửa artifact đã sealed chỉ để làm gate PASS.

Không tự thay đổi execution semantics.
Không tự chọn mã thay thế cho `BUY_FAILED` / `SELL_FAILED`; không tự tạo Cash từ `SELL_FAILED`; thiếu Cash phải xử lý theo scaling rule đã khóa.

Không làm mất ranh giới evaluation.
Development, Holdout và Final Evaluation phải giữ đúng segment boundary; không dùng Holdout hoặc Final để tune lại strategy. Sharpe chỉ được sử dụng ở segment được decision/config cho phép.

Không dùng artifact sai vai trò.
Official run phải sử dụng canonical/versioned inputs và outputs theo manifest.

Dashboard không được trở thành research engine hoặc investment recommender.
M3-B chỉ đọc và trình bày versioned/sealed artifacts; không fit model, không recompute methodology, không tự thực hiện PIT/ranking và không biến cluster label thành khuyến nghị mua/bán.

---

33. ĐIỀU KIỆN HOÀN THÀNH M3-A

M3-A chỉ hoàn tất khi:

- input Market/Financial/PIT/prices/calendar/VNINDEX đã được version/hash;
- các decision ảnh hưởng target/return đã khóa;
- config official là `LOCKED`;
- missing-price/BUY_FAILED/SELL_FAILED/scaling khớp semantics GitHub đã chốt;
- turnover khớp `(BUY+SELL)/(2×pre_trade_NAV)`;
- Development và Holdout đã chạy/seal riêng;
- mỗi official run có đủ 8 loại output canonical;
- `manifest.json` đủ để truy code/config/input provenance;
- benchmark/metrics khớp exact segment;
- Sharpe không xuất ở Development/Holdout;
- Final Evaluation chỉ chạy sau readiness + unseen audit và exact-window lock.

34. ĐIỀU KIỆN HOÀN THÀNH M3-B

M3-B hoàn tất khi:

- schema hiển thị đã có phiên bản;
- gói dữ liệu hiển thị bất biến;
- nếu dashboard đọc M3-A thì chỉ đọc sealed/versioned M3-A artifacts theo `manifest.json`;
- backend hiện hành sử dụng `web/backend` với FastAPI theo mô hình read-only;
- frontend hiện hành sử dụng `web/frontend` với Next.js;
- frontend/backend không fit model, không recompute research methodology và không tự tạo một PIT/ranking implementation riêng;
- kiểm thử API đạt;
- frontend build/test cần thiết đạt;
- màn hình tổng quan phân biệt rõ Market và Financial là hai hệ clustering độc lập;
- thời điểm Market observation và Financial PIT/usable_from_date được thể hiện đúng semantic, không

làm người dùng hiểu hai loại dữ liệu có cùng thời điểm;

- màn hình dịch chuyển/lịch sử tôn trọng tần suất cập nhật khác nhau giữa Market và Financial;
- màn hình chi tiết cho thấy lịch sử và provenance cần thiết;
- trạng thái missing/unavailable/stale hiển thị rõ, không biến thành zero;
- nếu hiển thị kết quả M3-A, phải truy được segment, run, config, input và provenance tương ứng;
- không có tính toán nghiên cứu trong dashboard;
- không biến cluster label hoặc strategy output thành khuyến nghị đầu tư;
- kiểm tra desktop/mobile đạt;
- truy nguyên từ source artifact → presentation bundle → API → frontend đạt.

Các implementation HTML/CSS/JavaScript hoặc dashboard legacy được repository giữ lại vì compatibility/history không được coi là active M3-B architecture và không được dùng để ghi đè `web/frontend` + `web/backend`.

35. ĐIỀU KIỆN HOÀN THÀNH M3-C

M3-C hoàn tất khi:

- 4 notebook hoặc bộ tương đương đọc đúng canonical output;
- Notebook 01 kiểm tra PIT bằng manifest + pinned inputs + official module, không cần alignment CSV legacy;
- Notebook 02 khớp target output;
- Notebook 03 khớp ledger/holdings/turnover/nav/benchmark/metrics;
- Notebook 04 chỉ báo cáo Final sealed run;
- turnover được kiểm tra theo:
- report không nhập tay số liệu và không phụ thuộc CSV intermediate đã bỏ;
- mọi kết quả truy được về `manifest.json`, config, code commit và input hashes.

---

36. TRẠNG THÁI HIỆN TẠI

Mục này chỉ theo dõi trạng thái decision/dependency.

Không định nghĩa methodology mới.

36.1. Methodology đã khóa / đã có owner decision

- Market và Financial là hai hệ clustering độc lập.
- Q1: A v4, k=3, là Market model chính thức; M2 K=2 giữ legacy/research control.
- Q2: ILLIQUID bị loại khỏi traded universe.
- Q3: Financial clustering chính thức = B-PIT; cutoff = max(30/04/(t+1), ngày usable theo Q-date), cluster order đóng băng tại cutoff (R1, R2, cần owner xác nhận).
- Q4: Financial sectors không có B-PIT coverage không vào Combined/Financial-only; không làm thu

hẹp Market-only/Equal-weight/Momentum-only.

- Q5: Market = leading cluster; Financial = not-weakest; Combined dùng Va.
- Q6: N_min = 10 cho Combined/Market-only/Financial-only.
- Q7: MC1 mom_126; MC2 downside_vol_63; MC3 liquidity_21.
- Q8: FC1 ROE; FC2 F-score; FC3 Z-score; partial missing → 0.5 + flag; all missing → exclude.
- Q9: Combined = 0.5 × S_market + 0.5 × S_fin; tie → liquidity → ticker.
- Financial runtime PIT = usable filter → max(fiscal_year).
- Q-date: FY>=2022 CafeF disclosure; FY<2022 lag sau 30/06 t+1; 30/04 sensitivity.
- Q10: stale = next FY usable hoặc 18 tháng sau period_end.
- Long-only, no leverage, monthly rebalance.
- Top share = 30%, ceil(0.30 × N), equal weight.
- Q11: main return basis = adj_close; VNINDEX = price index; price-only NOT_READY không block main.
- Q12: volume0 blocks both; ceiling blocks BUY; floor blocks SELL.
- BUY_FAILED/SELL_FAILED/scaling contract đã khóa.
- Q13: one-way turnover = (executed BUY + executed SELL)/(2 × pre_trade_NAV).
- Q14: VNINDEX benchmark; research controls = Market-only, Financial-only, Equal-weight, Momentum-only. M2 K=2 chỉ báo cáo số ứng viên (R7, cần owner xác nhận).
- Q15: Holdout sạch về portfolio-rule/return-based evaluation, không tuyên bố hoàn toàn unseen về upstream model-selection history.
- Q16: Final bắt đầu từ snapshot cuối 01/2026; exact end khóa sau readiness trước performance; Sharpe chỉ ở Final.
- Q17: adjusted-price one-session glitch dùng derived repair + flag; source immutable.
- Strict segment isolation: Target boundary vẫn lưu; execution vượt segment_end bị chặn.
- Missing valuation price dùng PRICE_STALE.
- Main cost/slippage = 0.
- Canonical outputs = targets/ledger/holdings/turnover/nav/benchmark/metrics/manifest.

36.2. Strategy universe đã khóa

```
Combined
→ Market Eligible ∩ Financial Eligible
→ Market leading ∩ Financial not-weakest

Market-only
→ Market Eligible Pool

Financial-only
→ Financial Eligible Pool ∩ traded universe

Equal-weight
→ traded universe

Momentum-only
```

```
→ traded universe ∩ mom_126 hợp lệ
```

Sector treatment:

```
Bank / Securities / Insurance

Combined
→ loại nếu không có Financial B-PIT state
→ FINANCIAL_SECTOR_NO_B

Financial-only
→ không dùng nếu Financial state bắt buộc không tồn tại

Market-only / Equal-weight / Momentum-only
→ không bị loại chỉ vì thiếu Financial B-PIT
```

36.3. Leading/allowed-cluster rule đã khóa

```
Market leading selector
→ median(mom_126) cao nhất
→ tie: median(mdd_126) cao hơn

Financial cluster order
→ median(ROE) cao hơn
→ tie: median(debt_to_equity) thấp hơn

Financial allowed
→ not_weakest

Financial cluster order
→ tính trên tập fit B-PIT tại cutoff(t), đóng băng trong artifact
```

Không suy semantic từ numeric label; ambiguity → fail closed.

36.4. Methodology đã khóa nhưng readiness/implementation còn phải hoàn tất

Không còn Q1–Q17/Q-date nào chờ mentor/owner duyệt về methodology.

Official full backtest còn phụ thuộc:

- B-PIT build + audit + pin PASS;
- docs/DECISIONS.md/ADR sync;
- Khóa toàn bộ các tham số cấu hình chính thức (`configs/experiments/m3_strategy_v1.json`):
  - `status = "LOCKED"` (chính thức cho official run kèm SHA-256 pin);
  - `financial_rule = "not_weakest"` (chính thức theo Q5);
  - `market_criteria` : `mom_126: true` , `downside_vol_63: false` (MC2, lower is better theo Q7),

`liquidity_21: true`;

- `financial_criteria`: `roe: true`, `f_score: true`, `z_score: true`;
- `costs_bps_one_way = [0]` (baseline run dùng cost = 0, slippage = 0; các kịch bản 15/30 bps chuyển sang phần mở rộng);
- `segments` và `segment_end_dates`: định rõ ranh giới phân đoạn (`segment_end_dates` Development = `"2023-12-29"`, Holdout = `"2025-12-31"`, áp dụng strict isolation theo Mục 3.9);
- `financial_model`: trỏ tới B-PIT artifact đã pin (version + sha256);
- `stale_months = 18` (đọc từ config runtime);

- strict segment-boundary code/test PASS (không để lệnh và định giá kỳ cuối Development tràn sang tháng 2/2024);
- Q12/Q17 execution-price tests PASS;
- M2 K=2: chỉ báo cáo số ứng viên theo kỳ (R7), không cần backtest path;
- `run_m3.py` ghi `universe_audit_<strategy>.csv` (Mục 16.10);
- tradability fail-closed khi thiếu dữ liệu trần/sàn (Mục 3.8.6);
- N_min tính sau khi loại `FINANCIAL_CRITERIA_MISSING` (Mục 3.5.3, 3.7);
- chạy lại feasibility với B-PIT trước LOCKED (Mục 31, Giai đoạn 3);
- runtime inputs PASS;
- output writer/manifest PASS (đồng bộ hash đầu vào và commit hash vào `manifest.json`);
- exact Final end date readiness lock trước Final run.

Không block main nhưng chưa READY:

```
secondary price-only
→ NOT_READY do chưa có corporate-action readiness
```

Phần mở rộng, không thuộc main blocker:

- DELISTED_EXIT
- cost sensitivity 15/30 bps

36.5. Dependency kỹ thuật trước official run

Official run yêu cầu:

- runtime inputs PASS;
- B-PIT READY;
- config/code/ADR sync PASS;
- config = LOCKED;
- config hash pinned;
- output writer PASS;
- holdings_<strategy>.csv đúng schema date/ticker/weight;
- manifest.json đúng schema GitHub hiện hành;

- execution/turnover/Q12/Q17 tests PASS;
- segment boundary PASS.

Luồng trạng thái:

```
OWNER DECISIONS LOCKED
      +
IMPLEMENTATION / CONFIG SYNC
      +
INPUT READINESS (bao gồm B-PIT)
      +
OUTPUT READINESS
      ↓
TARGETS-ONLY AUDIT
      ↓
LOCKED CONFIG
      ↓
DEVELOPMENT
      ↓
VERIFY / SEAL
      ↓
HOLDOUT
      ↓
VERIFY / SEAL
      ↓
FINAL READINESS + WINDOW LOCK
      ↓
FINAL EVALUATION
```

37. KẾT LUẬN

M3 giữ Market Clustering và Financial Clustering là hai hệ upstream độc lập; việc kết hợp chỉ diễn ra tại tầng strategy/portfolio.

M3-A sử dụng các state upstream đã được tạo, áp dụng đúng PIT, eligibility, strategy, ranking và execution contract đã được khóa để tạo và backtest danh mục.

Mỗi strategy phải sử dụng đúng universe và rule đã khóa của chính strategy đó. Nếu config/code Git còn PROPOSED hoặc chưa sync, đó là implementation/config blocker và không được dùng cho official return backtest.

M3-B là read-only product/presentation layer; không fit lại model và không recompute research methodology.

M3-C tái lập, kiểm chứng và báo cáo kết quả từ đúng input, config, code version và sealed artifacts của M3.

Development, Holdout và Final Evaluation phải tuân theo evaluation/readiness contract đã định nghĩa ở các mục tương ứng.

Chi tiết methodology, input contract, output contract, execution semantics, decision status, trình tự triển

khai và acceptance criteria được định nghĩa tại các mục canonical phía trên.

Mục 37 chỉ tổng kết phạm vi M3 và không tạo thêm methodology, rule hoặc decision mới.